

SlimPipe: Memory-Thrifty and Efficient Pipeline Parallelism for Long-Context LLM Training

Zhouyang Li*
Kuaishou Technology
Beijing, China
lizhouyang@kuaishou.com

Yuliang Liu*[†]
Kuaishou Technology
Beijing, China
liuyuliang@kuaishou.com

Wei Zhang
Kuaishou Technology
Beijing, China
zhangwei45@kuaishou.com

Tailing Yuan
Kuaishou Technology
Beijing, China
yuantailing@kuaishou.com

Bin Chen
Kuaishou Technology
Beijing, China
chenbin08@kuaishou.com

Chengru Song
Kuaishou Technology
Beijing, China
songchengru@kuaishou.com

Abstract

Pipeline Parallelism serves as a crucial technique for training Large Language Models, as it alleviates memory pressure from model states with relatively low communication overhead. However, in long-context scenarios, existing pipeline parallelism methods fail to address the substantial activation memory pressure, primarily due to the peak memory consumption resulting from the accumulation of activations across multiple microbatches. Moreover, these approaches inevitably introduce considerable pipeline bubbles, further hindering efficiency.

To tackle these challenges, we propose *SlimPipe*, a novel approach to fine-grained pipeline parallelism that employs uniform sequence slicing coupled with 1F1B schedule. This method constrains the activation memory accumulation to a single microbatch at any stage, by partitioning the sequence into smaller slices. Although the slices are evenly partitioned, the computation cost is not equal across slices due to causal attention. To counter this, we introduce a sophisticated workload redistribution technique to balance the computation. Consequently, SlimPipe achieves two key benefits simultaneously: (1) *near-zero memory overhead* and (2) *minimal pipeline bubbles*. For a 512K context length, SlimPipe boosts Model FLOPS Utilization (MFU) by up to 1.57× compared to state-of-the-art methods. More notably, it sustains over 45% MFU on 256 GPUs for an extreme context length of 2048K.

CCS Concepts

• **Computing methodologies** → **Parallel algorithms**; *Neural networks*.

*Both authors contributed equally to this research.

[†]Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SC '25, St Louis, MO, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1466-5/25/11

<https://doi.org/10.1145/3712285.3759855>

Keywords

distributed deep learning, large language model, pipeline parallelism

ACM Reference Format:

Zhouyang Li, Yuliang Liu, Wei Zhang, Tailing Yuan, Bin Chen, and Chengru Song. 2025. SlimPipe: Memory-Thrifty and Efficient Pipeline Parallelism for Long-Context LLM Training. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 20 pages. <https://doi.org/10.1145/3712285.3759855>

1 Introduction

Autoregressive large language models, such as LLaMA [49, 51], Gemini [47], GPT [6], and Mixtral [19], have achieved dominant performance in many NLP tasks. Concurrently, an increasing number of studies [4, 9, 28] have found that context length plays a significant role in model capabilities, enabling applications like multi-turn dialogue and long-document understanding. However, as the model size and context length continue to grow, the storage requirements far exceed the capacity of modern accelerators. This memory demand comprises two primary components: (1) *model states*, including parameters, gradients, and optimizer states, which scale proportionally with model size, and (2) *activations*, whose memory footprint grows linearly with context length.

Pipeline Parallelism has become an indispensable component in hybrid parallelization strategies for training LLMs due to its ability to mitigate memory pressure from model states while maintaining relatively low communication overhead. However, existing PP approaches exhibit significant limitations in handling the memory pressure caused by activations, which scales with context lengths. As illustrated in Figure 1, while the memory footprint of model states decreases as the PP size increases, the activation memory remains constant. This limitation poses substantial challenges for exploring long-context models. For instance, as shown in Figure 2, even with maximal TP utilization within NVLink domains, the maximum viable context length for LLaMA 13B models is limited to 124K. Further context length expansion requires either memory-computation trade-offs through activation recomputing or sequence partitioning across nodes with low communication bandwidth, both of which significantly compromise training efficiency.

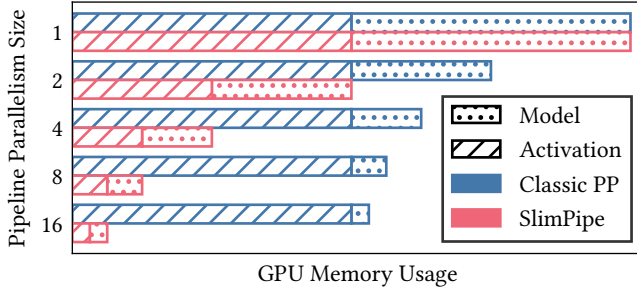


Figure 1: Comparison of GPU memory usage between Classic PP and SlimPipe. While both approaches consume identical GPU memory for model parameters, SlimPipe’s activation memory decreases proportionally as PP size increases, in contrast to Classic PP’s constant activation memory usage.

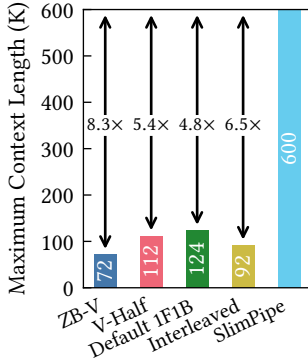


Figure 2: Maximum context lengths supported by different PP schemes in training Llama 7B with 8-way TP and 8-way PP.

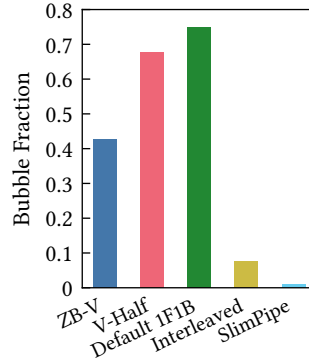


Figure 3: Bubble fractions of different PP schemes (PP size 8) in training Llama 13B with 4 microbatches and a 256K context length.

Beyond memory constraints, pipeline bubbles present another critical challenge in long-context scenarios. Both the 1F1B schedule [12, 32] and its interleaved variant introduce *warm-up bubbles* due to the necessity of forward warm-up phases to fill the pipeline and corresponding backward cool-down phases before synchronization. The small global batch sizes typical of long-context scenarios amplify the performance impact of these bubbles. ZB-V [38, 39] addresses this issue by decoupling the computations of the activation gradient (T_b) and the weight gradient (T_w) in the backward pass. When $T_f = T_b = T_w$, where T_f represents the forward computation time, ZB-V can achieve zero bubble. However, the time differences among T_f , T_b , and T_w can be substantial. For instance, the core attention operation, which lacks weights, implies $T_w = 0$, while T_b is also significantly larger than T_f . This difference introduces *imbalance bubbles*. The issues related to the imbalance bubbles of ZB-V will be thoroughly analyzed in the background and related work section. The current pipeline bubble characteristics are illustrated in Figure 3.

The fundamental limitation stems from the inherent nature of the classic PP design. Our key observation is that to achieve optimal throughput, the pipeline must accumulate p computational units during the warm-up phase, where p is the PP size. However, since existing PP methods operate at *microbatch-level* granularity, these methods require storing p microbatches of activations simultaneously. Although the number of layers per pipeline stage is reduced by a factor of p , the activation memory consumption remains constant due to the accumulation of p microbatches. Inspired by TeraPipe [26], we introduce a fine-grained 1F1B schedule. Our approach partitions the computation of each microbatch into smaller, equal-length units along the sequence dimension, introducing a processing granularity we term *slice-level*. This fine-grained decomposition significantly reduces the peak memory required for accumulated activations. As illustrated in Figure 1, this method maintains a stable and lower memory footprint during the pipeline’s steady state.

However, because mainstream LLMs employ causal attention, even though the slices are of equal length, the later slices impose higher computational cost than the earlier ones. This leads to pipeline imbalance bubbles. To counteract this, we design a novel method that equalizes memory usage and computational workload across different slices by redistributing the attention workload among different pipeline parallelism devices. The load imbalance with uniform slicing is primarily due to the attention calculation, so we redistribute an appropriate portion of the attention operation from heavily loaded devices to those with lighter load. This redistribution enables the less burdened devices to handle the additional attention computations and then send the outcomes back to the original devices. By employing this method, we maintain an equilibrium of computational and memory load across all uniform slices. This means that imbalance bubbles are completely eliminated. Moreover, in LLM training, to maintain model performance, the global token size is constrained by the critical batch size [29]. Longer sequences reduce the batch size and inflate the pipeline bubble overhead. By operating at slice-level granularity instead of microbatch-level, we can significantly reduce the pipeline warm-up bubbles. To summarize, we address pipeline bubble issues through two key mechanisms: (1) context redistribution that eliminates imbalance bubbles, and (2) fine-grained scheduling that reduces warm-up bubbles, collectively achieving near-zero pipeline bubbles, as depicted in Figure 3.

To this end, we introduce **SlimPipe**, a novel pipeline parallelism system designed for long-context training. SlimPipe integrates fine-grained uniform sequence slicing with the 1F1B schedule to achieve superior efficiency. It delivers three key advantages: (1) **near-zero memory overhead** from activations, (2) **minimal pipeline bubbles**, and (3) **enhanced computational efficiency**, as its memory-saving design reduces the need for activation recomputation. Our evaluations on various models and context lengths demonstrate that SlimPipe can boost the LLM training throughput to up to 1.57× compared to state-of-the-art systems.

In summary, our paper presents the following contributions:

- (1) Our analysis reveals two fundamental limitations in current pipeline parallelism approaches: memory inefficiency and

Table 1: Notations with description.

Notation	Description
L	number of transformer layers
M_h	size of one embedding tensor
M_a	total size of activations per microbatch
t	tensor parallelism size
c	context parallelism size
d	data parallelism size
e	expert parallelism size
p	pipeline parallelism size
m	number of microbatches
n	number of slices per input sequence
v	number of stages per pipeline device

significant pipeline bubble overhead. Moreover, we demonstrate that these issues exacerbate their difficulties in long-context training.

- (2) We propose SlimPipe, a novel approach to fine-grained pipeline parallelism that employs uniform sequence slicing coupled with 1F1B scheduling.
- (3) We propose a novel workload redistribution technique. By dynamically reallocating the attention workload between PP devices handling different slices, we achieve a balanced distribution of both computational and memory load among them.
- (4) Our extensive evaluation demonstrates that SlimPipe consistently outperforms state-of-the-art systems, such as DeepSpeed [17] and Megatron-LM [33, 45], achieving significant improvements in both memory utilization and throughput.

2 Background

The notations used in this paper are listed in Table 1.

2.1 Intra-operator Parallelism

Data Parallelism (DP) [53] involves replicating the model across multiple devices, with each device processing a different subset of the data. The gradients are then aggregated across all devices to update the model parameters. Data parallelism is effective for scaling out the training process by distributing the dataset.

Tensor Parallelism (TP) [45] is achieved by partitioning the parameters and inputs within an operator, effectively splitting the computational burden across multiple devices. This is particularly useful when the model is too large to fit into the memory of a single device. However, due to the significant communication overhead of tensor parallelism, it is generally confined within a node. **Sequence Parallelism (SP)** [22], as explained in the Megatron-LM framework, ensures that data remains in a partitioned state throughout the model training process, rather than maintaining redundant activations outside the attention and MLP modules.

Context Parallelism (CP) is an advanced technique used to train LLMs by distributing the sequence across multiple devices. *DeepSpeed Ulysses* [17] is a system optimization designed to enable the

training of extremely long-context transformer models. *Ring Attention* [5, 27] is an approach that leverages blockwise computation of attention to distribute long sequences across multiple devices.

Expert Parallelism (EP) [14, 16, 24, 40] is a technique utilized in Mixture-of-Experts (MoE) [13, 19, 55] architectures. In an MoE layer, different experts are dynamically activated according to the input tokens. EP distributes these specialized experts across multiple devices, enabling efficient computation by activating only the relevant experts for each input.

Table 2: Comparison between pipeline schemes.

Pipeline Scheme	Activation Memory	Bubble Fraction
GPipe	$\frac{m}{p}$ $\uparrow\uparrow$	$\frac{p-1}{m}$ $-$
TeraPipe	$\frac{m}{p}$ $\uparrow\uparrow$	$\frac{p-1}{nm}$ $\downarrow\downarrow$
PipeDream-Flush (default 1F1B)	1 $-$	$\frac{p-1}{m}$ $-$
Interleaved 1F1B	$1 + \frac{p-1}{vp}$ $\uparrow$	$\frac{p-1}{vm}$ $\downarrow\downarrow$
ZB-V	1 $-$	$\left(0, \frac{2(p-1)}{3m}\right)$ $\downarrow$
V-Half	$\frac{1}{2} + \frac{1}{p}$ $\downarrow$	$\left(\frac{p}{2m}, \frac{1}{3} + \frac{p}{2m}\right)$ $\uparrow$
SlimPipe	$\frac{1}{p} + \frac{2(p-1)}{nvp}$ $\downarrow\downarrow\downarrow$	$< \frac{p-1}{nom}$ $\downarrow\downarrow\downarrow$

2.2 Pipeline Parallelism

Pipeline Parallelism [12, 15, 20, 26, 31–33, 38, 39] splits a model across its layers, assigning distinct layers to different devices for simultaneous processing. However, this method can result in bubbles, which are caused by uneven workload distribution or periods of inactivity while waiting for gradient synchronization. To mitigate the problem of bubbles in pipeline parallelism, several studies have investigated solutions focusing on asynchronous updates. However, it carries the potential risk of affecting model performance. In this paper, we only consider synchronous pipeline parallelism.

Memory Consumption. GPipe [15] requires accumulating the activations for all microbatches until the backward pass is completed for the first microbatch, leading to high peak memory consumption. TeraPipe [26] exploits the autoregressive nature of LLMs through its token-level scheduling, but it inherits GPipe’s critical memory limitation: accumulating all activations throughout the pipeline.

PipeDream-Flush [32] and DAPPLE [12] introduces the 1F1B schedule, which divides each iteration into three phases: warm-up phase, steady phase, and cool-down phase. The warm-up phase aims to bring all pipeline devices into a working state. To ensure the availability of backward operations, activations generated during the warm-up phase are accumulated. Subsequently, all pipeline devices enter the 1F1B state during the steady phase, where the number of activations consumed by backward is equal to the number of activations produced by forward, thus maintaining stable

memory usage. This alters the accumulation from m microbatches to p microbatches. Seq1F1B [2] combines the 1F1B schedule with TeraPipe’s token-level schedule. However, the new schedule can no longer maintain stable memory usage due to its non-uniform slicing. None of these schedules can reduce the activation memory thoroughly. In contrast, SlimPipe attains **near-zero memory overhead**, with memory consumption scaling inversely with p .

Zero Bubble Pipeline Parallelism [38, 39] introduces a novel V-shape pipeline schedule that effectively balances memory utilization across pipeline devices. Through the V-shape schedule, ZB-V maintains the same peak memory overhead as 1F1B, while V-Half and V-Min reduce the peak memory to 1/2 and 1/3 of that of 1F1B, respectively. However, the activation memory consumption remains a constant that does not decrease with increasing p .

Bubble Fraction. GPipe treats the entire computation of a microbatch across all layers in a pipeline stage as an atomic *computation unit*. This coarse granularity results in the pipeline stages experiencing significant warm-up bubbles.

To address the issue of bubble overhead, several works have approached the problem from both the model and input data perspectives. Instead of treating the entire batch as the computation unit, TeraPipe breaks down the computation into smaller units at the token level. This approach allows for more fine-grained scheduling and reduces the idle periods between pipeline stages. On the other hand, interleaved 1F1B [33] builds upon the 1F1B schedule and approaches the problem from the model perspective. In interleaved 1F1B, each device can undertake computations for several discrete subsets of layers rather than a single continuous sequence of layers. SlimPipe synergistically combines fine-grained model and input data sharding to **minimize warm-up bubble overhead**. To address workload imbalance caused by causal attention among slices, we propose a novel context redistribution mechanism that dynamically balances computational load across pipeline stages, effectively **eliminating imbalance bubbles**.

ZB-V [39] innovatively splits backward into two parts: one for input gradient and the other for parameter gradient, theoretically eliminating bubbles. However, assuming the forward, input gradient, and parameter gradient computation costs for a microbatch are T_f , T_b , and T_w , respectively, bubbles can only be avoided if $T_f = T_b = T_w$, which is impossible for Transformer models with attention blocks. For attention block, theoretically, $T_b = 2 \times T_f \gg T_w = 0$. When accounting for modern optimizations like Flash Attention and the inherent MFU disparity between forward/backward passes, the situation further deteriorates. Given that the computational complexity of attention is quadratic with respect to context length, the attention component tends to dominate the entire training process in long-context scenarios. This leads to significant challenges for ZB-V in long context training.

In summary, SlimPipe outperforms other pipeline parallelism methods in terms of memory efficiency, achieving near-zero memory overhead. It also demonstrates excellent performance in reducing pipeline bubbles. The results are presented in Table 2. It is worth emphasizing that in long-context training, SlimPipe’s advantages become even more pronounced.

2.3 Activation Rematerialization

Activation Checkpointing [7, 21] is a memory optimization technique that reduces memory usage by strategically releasing activation. When applied across a sequence of layers, this method retains only the initial layer’s input for the backpropagation. During the backward pass, intermediate outputs are recalculated as needed for gradient computation. This strategy significantly reduces the memory footprint of activations, thereby freeing up memory resources to accommodate larger models. Various instruments [3, 18, 35] automate the decision-making process, fine-tuning strategies to optimize runtime efficiency while staying within a predefined memory limit. Meanwhile, a recent work [54] introduces the concept of the Pareto frontier for activation checkpointing strategies in the context of LLM training. According to this approach, the strategy is developed by training models along the Pareto frontier, optimizing the trade-off between memory consumption and runtime efficiency.

Activation Offloading [43, 46], similar to activation checkpointing, is another memory optimization technique. It achieves memory savings by swapping activations that are temporarily not needed from device memory to host or NVMe memory, and then swapping them back to device memory when they are required.

3 Challenges in Long Context LLM Training

Immense Memory Overhead. Current training of long-context LLMs faces substantial memory pressure, which drives the need for hybrid parallelism. The memory pressure consists of model states due to the large model size and activations due to the long context. For instance, consider a 70B LLaMA model with a 1M context length. The memory for its activations is calculated by multiplying the context length (1,048,576), hidden dimension (8,192), number of layers (80), and bytes per value (2 for *bfloat16*). Even when distributed across a tensor parallelism size of 8, the required memory per device is approximately 160 GiB. This is double the 80 GiB memory available on mainstream GPUs like the NVIDIA H100, rendering standard training infeasible.

Limited Global Batch Size. Long context LLMs training maintains a relatively fixed global tokens size per iteration. This practice is to ensure the model performance. The concept of critical batch size has been introduced by [29]. Exceeding this size by inflating the global tokens size can result in excessive noise reduction, thereby adversely affecting the model’s performance. This means that as the context length continues to increase, the global batch size will scale inversely. The reduction in global batch size directly decreases the number of microbatches (m), as evidenced by Table 2, which proportionally increases the warm-up bubble overhead.

Imbalanced Model Partition. Recent LLMs feature exceptionally large vocabularies. For instance, LLaMA 3 [11] uses a vocabulary of 128K tokens, while Gemma [48] expands this to 256K. Conventional PP schemes typically colocate the large embedding, which handle this vocabulary on a single device, often the last stage. This naive partitioning strategy creates a severe imbalance, leading to both excessive memory consumption and a disproportionate computational load on that single device compared to others in the pipeline.

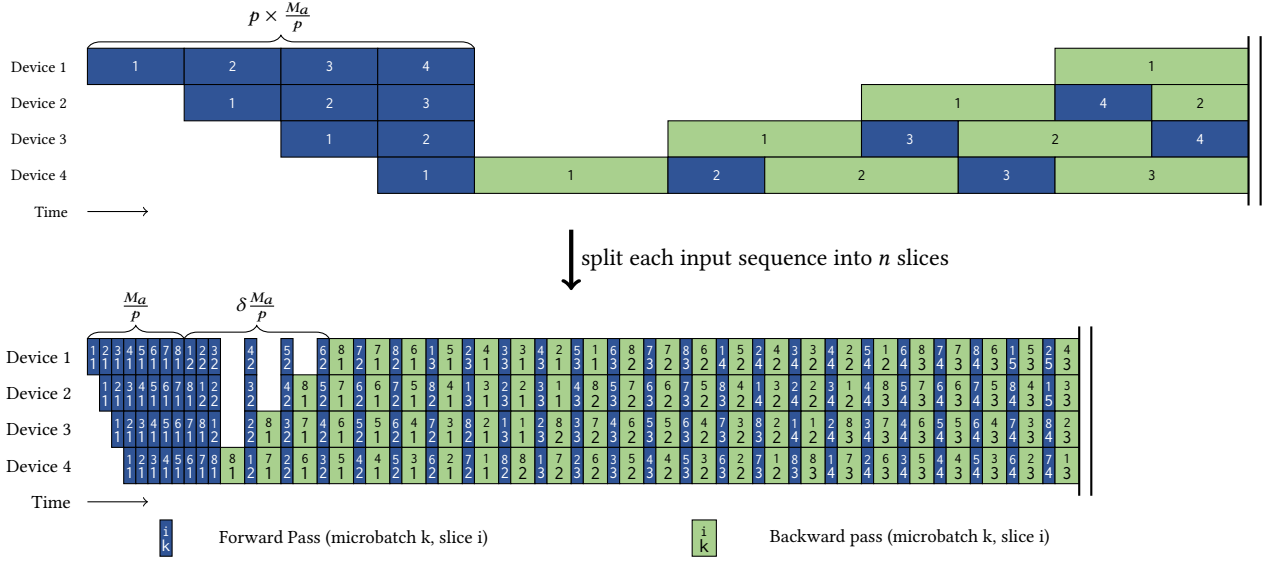


Figure 4: The top figure shows the default 1F1B schedule. The bottom figure shows the SlimPipe schedule, where each microbatch is split into n (in this case, 8) slices. M_a is the total size of activations per microbatch, which is distributed among p devices. The activations accumulated on device 1 reduces from M_a to $(1 + \delta) \frac{M_a}{p}$, where $\delta = 2(p - 1)/n$. And the warm-up bubble shrinks by about n times.

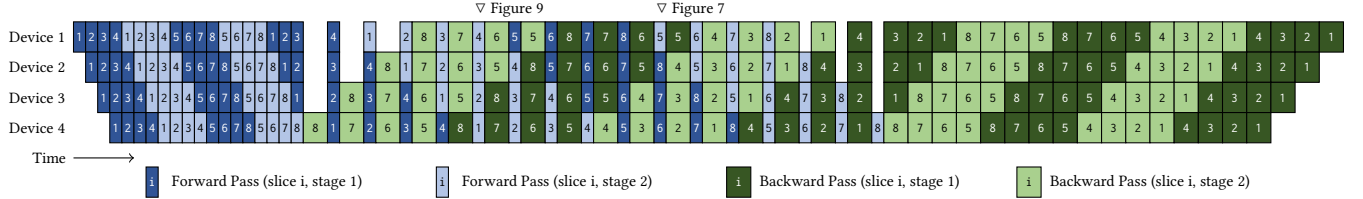


Figure 5: SlimPipe in its interleaving form. Each device is assigned 2 stages. Dark colors show the first stage and light colors show the second stage. For simplicity, only the slice number is labeled in the box and the microbatch number is omitted. Note that there are only two microbatches in this pipeline, each split into 8 slices. Whereas in the classic interleaved 1F1B pipeline, at least 4 (the PP size) microbatches are required. More details are depicted in Figure 7 and Figure 9.

4 Methods

In this section, we delve into the particular methods used by SlimPipe. First, we introduce how fine-grained pipeline parallelism with *uniform slicing* saves both memory and warm-up bubble time. We also provide a theoretical analysis to reveal the characteristics of our newly designed pipeline scheme. Next, we discuss the issue of workload imbalance among pipeline devices due to uniform slicing and causal attention. After that, we demonstrate how *attention context exchange* addresses the issue of workload imbalance. Finally, we employ *vocabulary parallelism* to equally distribute the computation and memory of the output layer among pipeline devices.

4.1 Pipeline Schedule with Uniform Slicing

4.1.1 Uniform Slicing. Pipeline Parallelism [15, 31] accumulates several forward passes on each device to warm up the pipeline.

These prefilled passes bring significant activation memory, especially on lower-rank devices of the pipeline. The long input sequence could be split into short slices to reduce the activation memory of each forward pass. And the straightforward approach to split a sequence is uniform slicing. However, the equal slice length will lead to unequal computation time between slices. The reason is the causal attention mechanism [26], by which a latter query attends to former keys and values, but not vice versa. We will discuss and solve this issue later in Section 4.2. Despite that issue, uniform slicing has the following substantial advantages over non-uniform slicing:

- (1) The maximum amount of accumulated memory is constrained.
- (2) The fixed slice length makes it more feasible for other techniques (e.g., CP) to divide at the sequence dimension.
- (3) Slices are prevented from being too short to maintain sufficient arithmetic intensity.

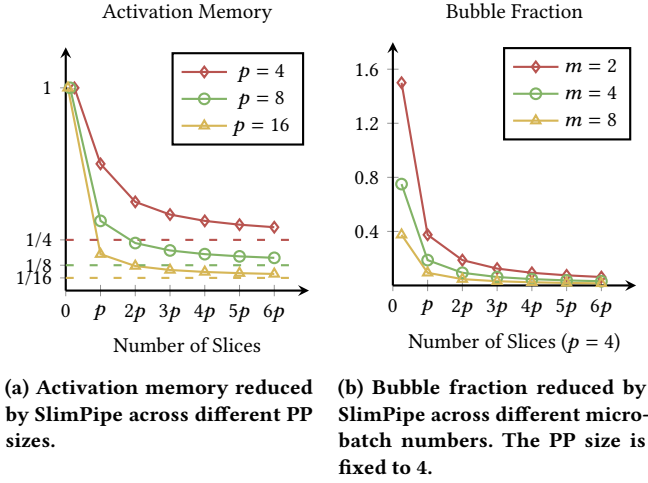


Figure 6: SlimPipe reduces both activation memory and bubble fraction. (a) The activation memories are equally large without uniform slicing (default 1F1B), and are decreasing from 1 towards $1/p$ separately as the number of slices increases. (b) The bubble fraction are quite large without uniform slicing (default 1F1B), and are decreasing to near-zero as the number of slices increases.

4.1.2 Slice-wise Scheduling. Firstly, we split the input sequence into n equal-length slices, where n is a multiple of p . To calculate the causal attention slice by slice, we deploy the KV cache technique [36]. Remarkably, the KV cache imposes no memory overhead on the accumulated activation. Because the keys and values are deliberately retained for gradient calculation in backward passes.

Next, we apply uniform slicing to the 1F1B pipeline scheme [31]. The 1F1B schedule is adopted with some vital adjustments. To perform backpropagation [44] for slices in one sequence, respectively, their backward passes are scheduled in reverse order of forward passes. Paired with this last-in first-out schedule, we release the corresponding portion of KV cache immediately once a backward pass is finished. This ensures that the crucial invariant of maximum accumulated memory is maintained through the 1F1B steady phase. In addition, we put more forward passes ahead to align forward and backward passes separately. The crafted pipeline scheme is depicted in Figure 4.

Furthermore, we smoothly extend our pipeline scheme to an interleaving form [33], by assigning multiple small stages to each device. As illustrated in Figure 5, uniform slicing and stage interleaving synergize well with each other. The accumulated activations and warm-up bubbles are further reduced.

4.1.3 Theoretical Analysis. As annotated in Figure 4, although the number of forward passes in the warm-up phase is increased, the total accumulated activation is actually decreased. Given that n is equal to or greater than p , the total accumulated activation size

$$M_{acc} = (1 + \delta) \frac{M_a}{p}, \quad \delta = \frac{2(p-1)}{n} \quad (1)$$

is much smaller than M_a . It is even approaching M_a/p as n continues increasing, as if virtually divided by the PP size. Moreover, the bubble time during warm-up and cool-down phases is reduced by about n times. In fact, the bubbles shrink super-linearly due to the causal attention mechanism. As a result, the bubble fraction is less than $\frac{p-1}{nm}$. When attention dominates the computation time (with an extremely long context length), the bubble fraction will become $\frac{(p-1)p}{(n+1)nm}$. Besides these benefits, the overall communication volume of default 1F1B has not been changed. The two significant characteristics of SlimPipe are depicted in Figure 6. With a fairly large value of n , the activation memory and bubble time are substantially reduced. In terms of the interleaving form, these two attributes will be further reduced by the number of stages v (formulas are listed in Table 2).

4.2 Attention Context Exchange

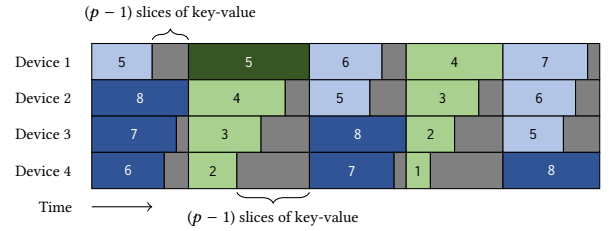


Figure 7: A closer look at the SlimPipe timeline with imbalance bubble exhibited. The beginnings of passes are aligned across devices to clearly reveal the bubbles' shape. The actual timeline will be more elaborated with propagated bubbles.

4.2.1 Imbalance Bubbles. If we just split each sequence into equal-length slices, the execution timeline will not be as ideal as depicted in Figure 5. Although we try to align each kind of passes across devices, bubbles arise and pervade between them, which is roughly sketched in Figure 7. These bubbles are caused by the uneven computation time of causal attention. As the query length is fixed by uniform slicing, the computation time is in proportion to the length of attended key-value. Latter slices, attending to more KV cache, cost more time to compute in both forward and backward passes.

At a specific moment, the workloads across pipeline devices conform to an arithmetic progression. And the difference between the heaviest and the lightest is $p-1$ slices of key-value. The situation becomes worse at the juncture of two microbatches, where the workload difference could be as great as $n-1$. Devices processing with less KV cache finish their computation sooner, and have to wait for other devices to communicate with them. This waiting-time is a type of imbalance bubbles. Imbalance bubbles will propagate along pipeline devices in both directions and cross with each other, making the pipeline timeline quite irregular.

4.2.2 Exchange the Context. To eliminate these messy imbalance bubbles, we need to make the attention workload as equal as possible across devices. The essential condition is that all devices at one moment have equal computation time. Note that it does not



Figure 8: The attention workloads are rebalanced by exchanging the context among pipeline devices. The local query and portions of the local key-value are sent to another device and the partial attention output will be sent back after the calculation being completed there.

require a uniform computation time across passes, which is otherwise required by TeraPipe. This essential condition allows us to redistribute the attention workload on the spot. As depicted in Figure 8, a device with more KV cache sends its query and portions of key-value to a device with less KV cache, and then the attention will be calculated there. The remotely calculated attention output will be sent back and merged with the locally calculated one via the online softmax method [30]. The attention workloads after redistribution become almost equal. The difference between them is at most one slice of key-value.

4.2.3 Communication Cost. We use the non-interleaving form of SlimPipe to demonstrate the communication volume of context exchange. The exchanged context is 1 slice of query (Q) and output (O) plus $\lfloor (p-1)/2 \rfloor$ slices of key (K) and value (V), and the latter becomes $\lfloor (n-1)/2 \rfloor$ slices at the microbatch juncture. The juncture period occupies $p-1$ out of n passes per microbatch. Generally, the unsliced query, key, value, and output all have the size M_h . So one slice of each from one device has the size $\frac{L}{p} \cdot \frac{M_h}{n}$. Finally, the total volume of exchanged context per microbatch per device is

$$\Theta = \left(\underbrace{2n}_{Q+O} + \underbrace{2(n-p+1)\left\lfloor \frac{p-1}{2} \right\rfloor}_{K+V, \text{ not at junctures}} + \underbrace{2(p-1)\left\lfloor \frac{n-1}{2} \right\rfloor}_{K+V, \text{ at junctures}} \right) \frac{LM_h}{pn} \quad (2)$$

$$\leq \left(2 - \frac{p-1}{n} \right) LM_h.$$

The inequality originates from the round-down operation. This volume is bounded by $2LM_h$, which is not related to the PP size and the number of slices. When applied to the interleaving form, context exchange gets a communication volume of the same degree.

We further reduce the overhead of context exchange by overlapping communication and computation. Specifically, a device transfers the remote attention context or results in parallel with its local attention computation. In principle, the attention computation time is quadratic with respect to the context length, while the communication time is linear. When the slice length is fairly long, the latency of communication can be covered thoroughly by the attention computation time.

4.3 Vocabulary Parallelism

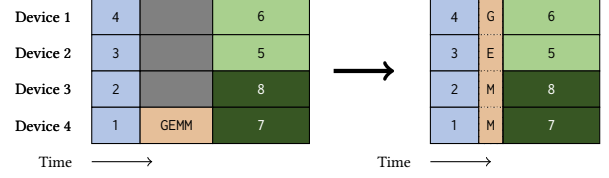


Figure 9: A SlimPipe timeline segment containing the output layer (GEMM). The left part shows the GEMM just assigned to the last device. The right part shows the GEMM distributed to all devices, where the bubble vanishes.

4.3.1 The Output Layer. Although we have balanced the memory consumption and computation workload in the previous sections, the pipeline parallelism still suffers from another sort of imbalance. The output layer of a language model is a GEMM operation that projects hidden states into the vocabulary space. As described in Section 3, the vocabulary size is really large. The output layer is calculated by the last pipeline device, thus significantly aggravating the workload imbalance here. As depicted in Figure 9, this workload imbalance also causes bubbles in the middle of the pipeline. Moreover, when training LLMs, the output layer is followed by a cross-entropy loss, which will store the vocabulary logits in *float32* for gradient calculation. That brings a large activation footprint to the last PP device. For instance, with a context length of 256K and a vocabulary size of 128,000, it consumes about 16 GiB memory of each GPU even in 8-way TP.

4.3.2 Distribute the Vocabulary. The point is to equally distribute the computation and memory of the output layer among pipeline devices. Specifically, we parallelize the GEMM along its vocabulary dimension (column-wise). The input hidden states are broadcasted to all PP devices. Thus, each device can calculate a part of the GEMM and get a portion of the vocabulary logits. Rather than gathering these portions together, we calculate the cross-entropy loss from the sharded logits, and the necessary statistics are synchronized. The scalar statistics are much smaller than the logits, so the communication volume is drastically reduced. Since the word embedding and the output layer usually share weights [37, 52], we also parallelize the former. This method was first adopted by the TP [45] implemented in Megatron-LM. However, it could not be trivially applied to common PP schemes due to their misaligned schedules. Since all passes are tidily aligned by attention context exchange, SlimPipe is particularly suitable for this method.

5 Implementation

We build SlimPipe within Megatron-LM [33, 45], and implement our functions using PyTorch [34]. We make some practical efforts to reduce the per-layer activations. We use the cuDNN [8] Scaled Dot Product Attention (SDPA) to calculate the attention. cuDNN SDPA is similar to FlashAttention [10], which avoids storing large intermediate matrices for backward. Our SwiGLU implementation recomputes the swish function instead of storing the intermediate activations. We also adopt a memory-efficient RMSNorm, which

otherwise uses its output to calculate gradients. Moreover, we customize the techniques as below, to better utilize them in SlimPipe.

5.1 Chunked KV Cache

The size of the KV cache grows up along the forward passes and shrinks down along the backward passes. We store the KV cache in individual tensors (chunks), instead of joining them together into an entire tensor. Because an entire tensor with variable size would frequently be re-allocated, leading to severe memory fragmentation issues. While PagedAttention [23] stores the KV cache in relatively small blocks, we store them in chunks just of the slice length to avoid block addressing in the GPU kernel. This is enough to eliminate memory fragmentation because all chunks have an identical size from uniform slicing. These chunks will be precisely reused during the 1F1B steady phase, where a backward pass releases one and a forward pass acquires one.

5.2 Variant Context Parallelism

Context parallelism is widely utilized to train LLMs with extremely long context. Existing CP implementations [5, 25, 27] communicate key-value shards among devices and use them to calculate the attention output against the local query shard. When CP is used with SlimPipe, the cached key-value will be communicated every time a later slice of query attends to them, which is rather inefficient. Therefore, we implement a variant of CP, which communicates the query, output and the normalizer in softmax instead of the key and value. These items will never be reused in later slices' attention, so the redundant communication is avoided. The calculation process of our variant CP is depicted in Figure 10. For concision, we adopt collective communication to implement it [11].

CP uses a ring-style point-to-point communication to cycle the key and value among devices. At least $c - 1$ transmissions are required by each device to loop over all data shards. So, the communication volume of CP is

$$\begin{aligned}\Theta_{cp} &= (c - 1) \frac{2M_h}{c} \\ &= 2 \frac{c - 1}{c} M_h.\end{aligned}\quad (3)$$

When using CP with SlimPipe, the cached key-value of the i -th slice will be communicated again in the following $(n + 1 - i)$ slices' attention. The communication volume thus increases to

$$\begin{aligned}\Theta'_{cp} &= \sum_{i=1}^n (n + 1 - i)(c - 1) \frac{2M_h}{nc} \\ &= (n + 1) \frac{c - 1}{c} M_h.\end{aligned}\quad (4)$$

The *all-gather* and *reduce-scatter* each requires $c - 1$ transmissions of one data shard, and the *all-reduce* requires $2(c - 1)$ transmissions of one data shard. The normalizer's size is only M_h/d , where d is the attention head dimension. Since the value of d is as large as 128 or 256 in common transformer models [6, 48, 50], we could have $M_h \gg M_h/d$. Therefore, the communication volume of this

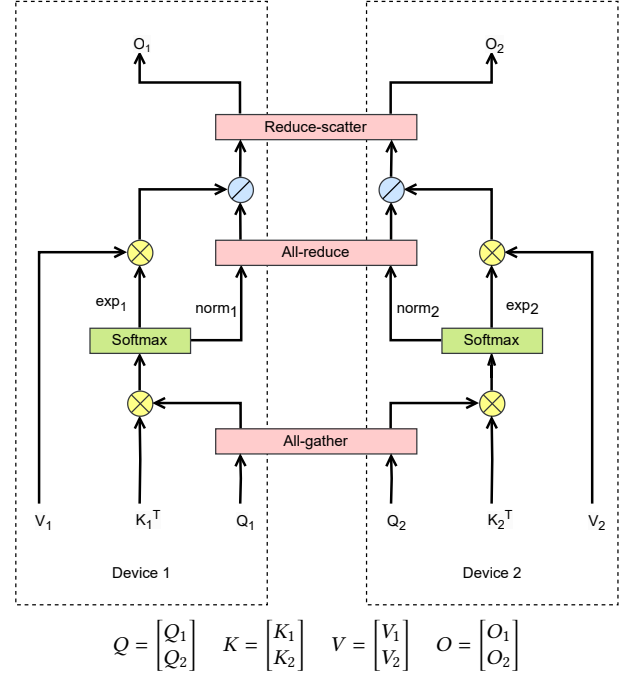


Figure 10: Two devices calculating attention in commutated context parallelism. The query (Q), key (K), value (V) and output (O) are sharded in the sequence dimension. *exp* and *norm* are the exponentials and the normalizers in *softmax*. The division $\oslash$ in *softmax* is deferred to after the matrix multiplication $\otimes V$.

CP variant is

$$\begin{aligned}\Theta''_{cp} &= n \left((c - 1) \frac{2M_h}{nc} + 2(c - 1) \frac{M_h/d}{nc} \right) \\ &= 2 \frac{c - 1}{c} (M_h + M_h/d) \\ &\approx 2 \frac{c - 1}{c} M_h,\end{aligned}\quad (5)$$

which is identical to that in Equation 3. That is to say, our variant CP is still efficient when used with SlimPipe.

6 Evaluation

6.1 Experimental Settings

Table 3: Models used in evaluation.

	a	number of attention heads	h	hidden dimension size		
	g	number of query groups	H	FFN dimension size		
Model	L	a	g	h	H	#Params.
Llama 13B	40	40	—	5120	13824	13.3×10^9
Llama 70B	80	64	8	8192	28672	69.5×10^9
Llama 149B	96	96	8	12288	32768	148.9×10^9
Mixtral 8x7B	32	32	8	4096	14336	47.0×10^9
Mixtral 8x22B	56	48	8	6144	16384	141.0×10^9

All experiments are conducted on a GPU cluster where each node is configured with two Intel Xeon Platinum CPUs and 1TB of memory. In each node, 8 NVIDIA Hopper 80GB GPUs are installed and interconnected via NVLink, whose bandwidth is 400 GB/s per GPU. Besides that, every GPU is coupled with a 400 Gbps NIC for inter-node communication. Unless otherwise stated, TP, CP and EP should be deployed within a node, while PP and DP could be deployed across nodes. TP is always paired with SP.

Models of various architectures are used in evaluation. We select the LLaMA-like dense models and the Mixtral series MoE models. The specifications of the models are listed in Table 3. GQA [1] is applied to models except LLaMA 13B. In MoE layers, 2 out of 8 experts are routed for each token. The expert router is set to complete balance for performance measurement. Every model is coupled with a vocabulary of 128,000 entries. The training precision is *bfloat16* except that *float32* is used in loss calculation and gradient accumulation. The optimizer is Adam with *float32* internal states.

6.2 Memory Saving

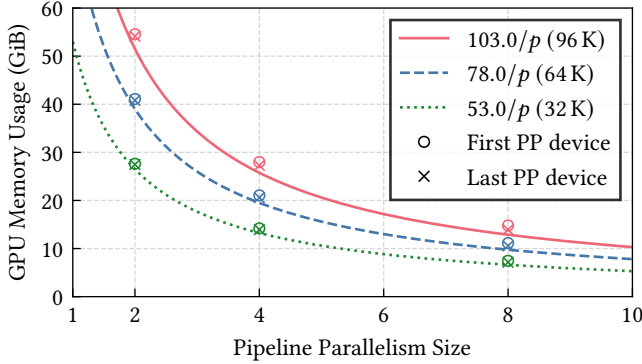


Figure 11: The memory reduced by the PP size p . The measured values align well with our theoretical model.

We use a simple experiment to show how SlimPipe reduces the memory usage in training. We train the LLaMA 13B model with input sequence lengths of 32K, 64K and 96K. The TP size is 8 and the PP size increases from 2 to 8. The number of stages per device is set to $v = L/p$, and the number of slices is set to $n = 4p$. We measure the memory usage of the first and the last pipeline devices, using the `torch.cuda.max_memory_allocated` method.

We draw the measured data as markers in Figure 11. We also draw a curve for each sequence length, whose formula is M_t/p , where M_t is the theoretical memory required to train the model without PP (only with 8-way TP). We use the memory modeling method from previous works [22, 54]. As we can see, the memory usage for both devices decreases in close alignment with the theoretical curves. The total memory is nearly in an inverse proportional relationship to the PP size, as if distributed by PP. While classic PP distributes model states, SlimPipe additionally distributes activations. In particular, the activations of the output layer are also distributed, as evidenced by our measurements on the last device. The memory usage of the first device is slightly higher than that of the last device, and the gap is formulated as $2(p-1)M_a/nvp$.

6.3 Insights into Slice Length

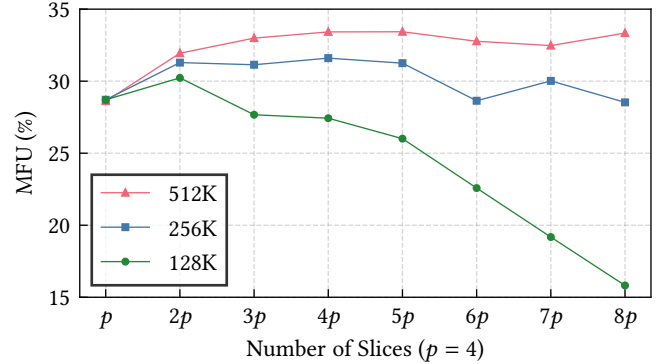


Figure 12: MFU of training the Llama 13B model with different context lengths. The PP size is 4 and the numbers of slices increases from p to $8p$.

As discussed in Section 4.1.1, fine-grained slicing can reduce the accumulated activations but also decrease the arithmetic intensity. So in this experiment, we will study how the value of n affects the training efficiency, and find an appropriate n for a given context length. We train the LLaMA 13B model with 2 microbatches per iteration. The three sets of input sequence lengths are 128K, 256K and 512K respectively. The parallelism configuration is 8-way TP combined with 4-way PP, and full checkpointing is enabled. The number of stages per pipeline device is 5, and the number of slices per sequence gradually increases from p to $8p$. We use MFU to quantify the training efficiency.

As we can see in Figure 12, fine-grained slicing improves MFU initially (due to reduced bubbles) but harms it eventually (due to lower arithmetic intensity) as slices become shorter. The MFU of 128K context length drops sharply after $n = 2p$, indicating that the arithmetic intensity is too weak. We also see that the transition point of n comes later for a longer context length. MFU of the 512K context length remains high even with 32 slices per sequence. In conclusion, we should choose a moderate value of n to maintain the training efficiency while saving memory.

6.4 End-to-End Performance

When SlimPipe is integrated into a hybrid parallelism training system, it could introduce new optimization opportunities to other techniques. For example, the GPU memory saving could avoid the use of full checkpointing, thus increasing the system efficiency. In the main course, we compare the system performance between Megatron-LM and SlimPipe. For clarity, we refer to the baseline system with interleaved 1F1B PP as Megatron-LM, and the system with our innovative PP as SlimPipe. Prevalent parallelism techniques (TP, CP, EP, DP, and PP) are organically composed in both systems. The activation saving techniques mentioned in Section 5 are applied uniformly, while full or selective checkpointing [22, 54] is enabled on their demand. The selective checkpointing implemented by us just recomputes the up projection plus SwiGLU in an MLP layer. DeepSpeed [42] is also involved and is equipped with its Zero

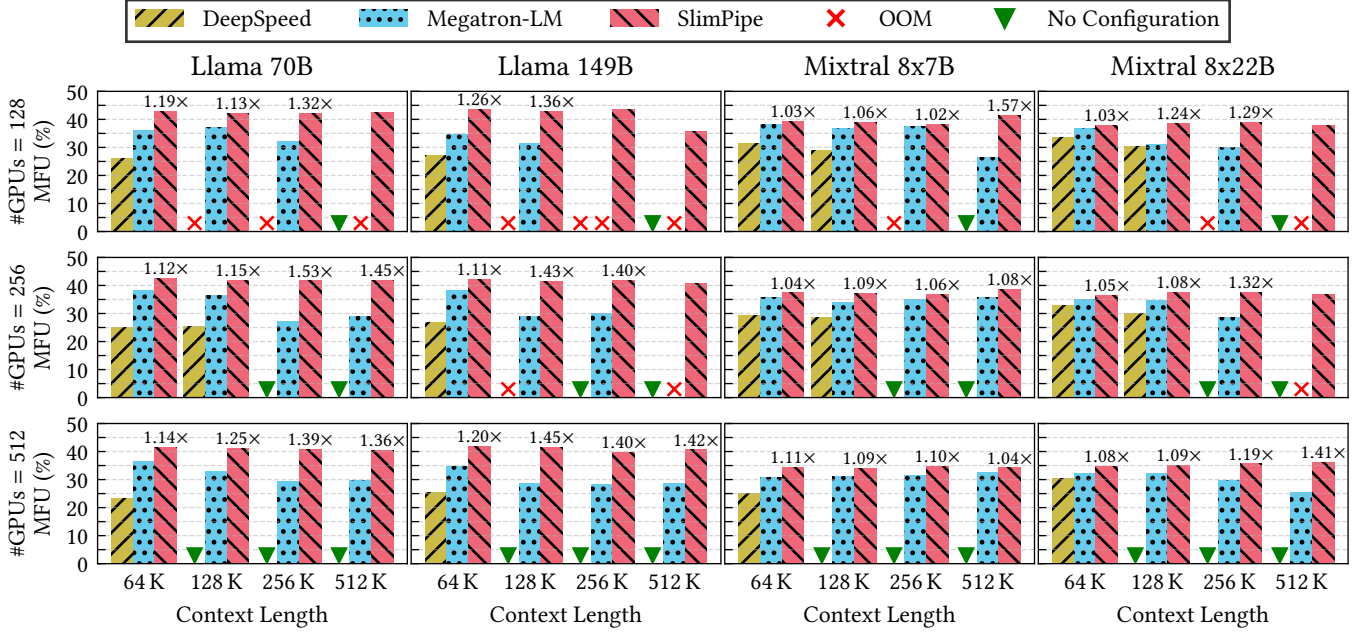


Figure 13: System performance comparison between DeepSpeed, Megatron-LM, and SlimPipe across different models and GPU counts. The annotations above the bars for SlimPipe represent its relative speedup over Megatron-LM. A green triangle marker indicates that no viable configuration could be found under this condition, and a red cross indicates that all candidate configurations result in Out-Of-Memory (OOM) errors.

Redundancy Optimizer [41] and Ulysses Parallelism (UP). To exhibit the best performance of each system, their hybrid parallelism configurations are baked through grid search.

We select 4 models covering dense and MoE architectures, in various sizes. The input sequence length extends from 64K to 512K. The per-iteration token size is fixed to 4M, which means that there are fewer microbatches when the sequence length is longer. The total number of GPUs scales from 128 to 512. The comprehensive benchmark results are shown in Figure 13. SlimPipe outperforms the other two systems in training efficiency through all models, context lengths, and number of GPUs.

Difference in Context Lengths. SlimPipe demonstrates increasingly significant advantages when training with longer context lengths. Extended contexts inherently demand greater activation memory consumption and incur more significant warm-up bubbles. As previously discussed, SlimPipe’s fine-grained schedule minimizes bubbles overhead, thereby delivering substantial efficiency benefits. Moreover, leveraging its memory-thrifty design, which substantially reduces activation storage requirements, we maintain training efficiency without resorting to full checkpointing or expensive cross-node TP/CP communication. In contrast, the other two approaches progressively adopt inefficient full checkpointing and expensive cross-node communication as context lengths grow, ultimately encountering out-of-memory failures in many cases. For example, in the 512K context length scenario with 128 GPUs, SlimPipe achieves a 1.57× speedup over Megatron-LM when training Mixtral 8x7B. Meanwhile, it maintains high efficiency on other larger models, while Megatron-LM encounters OOM issues.

Difference in Scalability. DeepSpeed fails to run with a 512K context length on a total of 128 GPUs (no viable configuration), because the batch size 8 is not enough for a larger DP size. It cannot enlarge the UP size because there are only 8 query groups. This limitation of DeepSpeed becomes severer if running on more GPUs. When scaling to 512 GPUs, systems have to expand their DP size to saturate this amount of GPUs. A larger DP size leads to fewer microbatches, and thus to a shorter steady phase in the pipeline. Consequently, a larger bubble fraction hinders the scalability of Megatron-LM. Furthermore, if the DP size continues expanding, the number of microbatches left in PP will become even less than the PP size, breaking the minimum requirements by the interleaved 1F1B scheme. This fatal limitation prevents Megatron-LM from scaling to more GPUs. In contrast, SlimPipe maintains quite high training efficiency with as few as 2 microbatches in PP, because it effectively reduces the warm-up bubbles by fine-grained slicing.

Difference in Models. SlimPipe’s advantages become increasingly pronounced with larger-scale models. For both dense and MoE architectures, SlimPipe demonstrates superior performance as model size grows. Crucially, SlimPipe eliminates the need for either full checkpointing or costly cross-node communication during large-model training. For example, in the 512 GPUs, 512K context length scenario, SlimPipe achieves a 1.41× speedup on the 8x22B model, while it only achieves a 1.04× speedup on the 8x7B model. This scalability advantage suggests that SlimPipe’s performance lead over alternative systems would further widen with even larger models.

In a nutshell, SlimPipe demonstrates its outstanding aptitude to train long-context LLMs at any scale.

6.5 Ultra Long Context

Table 4: System performance of training 4 LLMs with the maximum supported context length, using at most 256 GPUs. The selective checkpointing is enabled uniformly while the offloading ratio is adaptive. The TPS metric means tokens per second per GPU.

Model	Context	<i>t</i>	<i>c</i>	<i>e</i>	<i>d</i>	<i>p</i>	<i>n</i>	TPS / MFU
Llama 70B	2048K	4	4	–	1	16	64	51 / 45.0%
Llama 149B	2048K	4	2	–	1	32	64	26 / 42.0%
Mixtral 8x7B	4096K	1	8	8	2	16	128	125 / 42.6%
Mixtral 8x22B	2048K	1	8	8	1	28	64	90 / 41.7%

512K context is not the very limit of SlimPipe. To explore the extensibility of it, we integrate the pipeline-parallelism-aware offloading [54] into our system. This technique saves GPU memory by transferring portions of the activations to the host memory. Our fine-grained pipeline schedule can provide more opportunities to offload and reload activations. We train 4 large models on 256 GPUs, with 16M tokens per iteration and extend the context length to the maximum supported by our system. The detailed system configurations and performance metrics are listed in Table 4. With the help of offloading, SlimPipe pushes the boundary of context length to an astonishing 4096K. While training with these spectacular context lengths, SlimPipe still maintains remarkably high efficiency.

6.6 Comparison of Pipeline Schemes

The last is a comparison between pipeline schemes on training LLMs with long context length. The model is LLaMA 13B and the batch size is 4. The input sequence length increases from 32K to 512K. 8-way TP and full checkpointing are used to reduce the memory pressure. The number of stages per device is set to 5 for both interleaved 1F1B and SlimPipe. The number of slices is fixed to 4 for SlimPipe. The ZB-V and V-Half use *float16* in place of *bfloat16*, for the latter is not yet supported.

The MFUs are drawn as dots in Figure 14, and the memory consumptions are drawn as bars in Figure 15. The default 1F1B consumes moderate memory and can sustain a fairly long context length of up to 256K. But its efficiency is quite low due to its large warm-up bubbles. The interleaved 1F1B shows competitive efficiency, because it has smaller warm-up bubbles than the default 1F1B. But its memory overhead confines it to a context length of 128K. The two V-shaped schemes (ZB-V and V-Half) decouple the backward passes of inputs and weights and rearrange them to fill up the warm-up and cooldown bubbles. We observe that they consume abnormally high memory even when training with a 32K context length. We find that the full checkpointing does not work properly in their decoupled backward passes. This fatal problem stops them from running with longer context lengths. Worse still, they both suffer from severe imbalance bubbles, caused by the divergent computation time of two decoupled backward passes. To be specific, the backward of inputs involves the attention backward, thus taking much longer time than the backward of weights. This kind of imbalance bubbles makes them quite inefficient. SlimPipe resolves all

imbalance issues and reduces memory usage and pipeline bubbles. Its memory usage is as low as expected and is, of course, lower than others at every context length. As a result, SlimPipe supports a context length as long as 512K, and delivers higher efficiency than others through all context lengths.

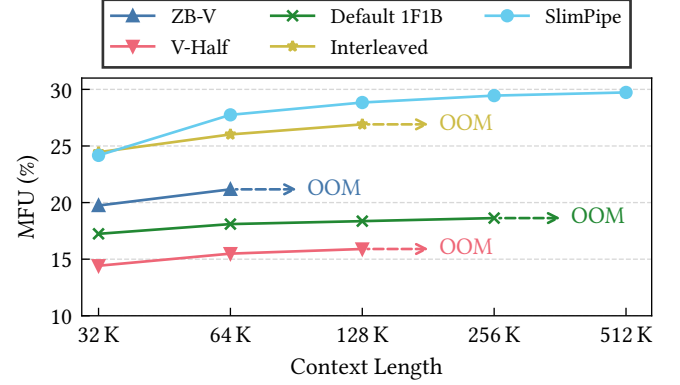


Figure 14: Comparison of MFU across different PP schemes.

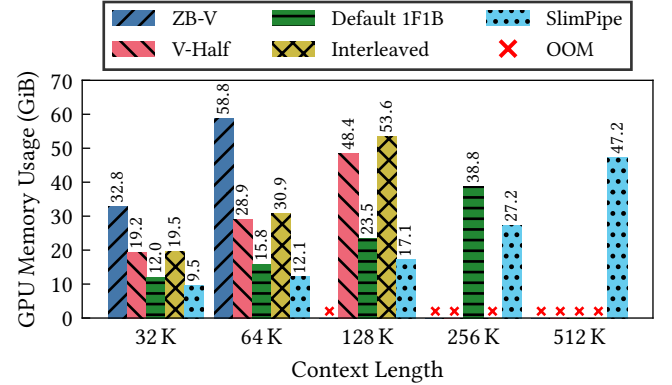


Figure 15: Comparison of GPU memory usage across different PP schemes.

7 Conclusions

This paper introduces SlimPipe, a fine-grained pipeline parallelism with uniform slicing. Firstly, SlimPipe leverages uniform slicing to reduce GPU memory usage for activations proportionally as the PP size increases. Additionally, this fine-grained pipeline parallelism schedule reduces bubbles during the warm-up and cool-down phases. Furthermore, we found that uniform slicing, due to the presence of causal attention, leads to workload imbalance between different slices. To avoid pipeline bubbles introduced by imbalanced workload, SlimPipe incorporates an attention context exchange mechanism. By utilizing the vocabulary parallelism, SlimPipe also gets rid of the workload imbalance caused by the output layer. Our experiments demonstrate that SlimPipe can outperform state-of-the-art training systems.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245* (2023).
- [2] Sun Ao, Weilin Zhao, Xu Han, Cheng Yang, Xinrong Zhang, Zhiyuan Liu, Chuan Shi, and Maosong Sun. 2025. Seq1F1B: Efficient Sequence-Level Pipeline Parallelism for Large Language Model Training. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 8998–9008. <https://aclanthology.org/2025.naacl-long.454/>
- [3] Olivier Beaumont, Lionel Eyraud-Dubois, Julien Hermann, Alexis Joly, and Alena Shilova. 2019. Optimal checkpointing for heterogeneous chains: how to train deep neural networks with limited memory. *arXiv preprint arXiv:1911.13214* (2019).
- [4] Amanda Bertsch, Maor Ivgi, Emily Xiao, Uri Alon, Jonathan Berant, Matthew R Gormley, and Graham Neubig. 2024. In-context learning with long-context models: An in-depth exploration. *arXiv preprint arXiv:2405.00200* (2024).
- [5] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. 2023. Striped attention: Faster ring attention for causal transformers. *arXiv preprint arXiv:2311.09431* (2023).
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, J. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, T. Henighan, R. Child, A. Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Ma teusz Litwin, S. Gray, B. Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, I. Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *Neural Information Processing Systems* (2020).
- [7] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [8] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759* (2014).
- [9] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. 2019. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860* (2019).
- [10] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
- [11] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab Al-Badawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalala, Lauren Rantala-Yearry, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Ross Hays, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoqiang Nie, Sharan Narang, Shathar Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stéphane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyan Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaoqing Ellen Tan, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Couderc, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aaron Gattafiori, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alex Vaughan, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Franco, Aparajita Saraf, Ankur Bhandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gammido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Damon Civin, Dana Beaty, Danny Kreymer, Daniel Li, Danny Wyatt, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomerly, Eleonora Presani, Emily Hahn, Emily Wood, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Firat Ozgenel, Francesco Caggioni, Francisco Guzmán, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Govind Thattai, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Karthik Prasad, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kun Huang, Kunal Chawla, Kushal Lakhotia, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Maria Tsimpoukelli, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Gebo, Nicolas Ununier, Nikolay Pavlovich Laptev, Ning Dong, Ning Zhang, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollár, Polina Zvyagina, Prashant Ratanachandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Rohan Maheswari, Russ Howes, Ruty Rinott, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Kohler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vitor Albiero, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaofang Wang, Xiaojian Wu, Xiaolan Wang, Xide Xia, Xilun Wu, Xinbo Gao, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yuchen Hao, Yundi Qian, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, and Zhiwei Zhao. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).
- [12] Shiqiang Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. 2021. DAPPLE: A pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 431–445.
- [13] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

- [14] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. Megablocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems* 5 (2023), 288–304.
- [15] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).
- [16] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. 2023. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems* 5 (2023), 269–287.
- [17] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509* (2023).
- [18] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. 2020. Checkmate: Breaking the memory wall with optimal tensor rematerialization. In *Proceedings of Machine Learning and Systems*, Vol. 2. 497–511.
- [19] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [20] Taebum Kim, Hyoungjoo Kim, Gyeong-In Yu, and Byung-Gon Chun. 2023. BPIPE: memory-balanced pipeline parallelism for training large language models. In *International Conference on Machine Learning*. 16639–16653.
- [21] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2020. Dynamic Tensor Rematerialization. *arXiv preprint arXiv:2006.09616* (2020).
- [22] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023), 341–353.
- [23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [24] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).
- [25] Shenggui Li, Fuzhao Xue, Chaifanya Baranwal, Yongbin Li, and Yang You. 2023. Sequence Parallelism: Long Sequence Training from System Perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2391–2404.
- [26] Zhuohan Li, Siyuan Zhuang, Shiyuan Guo, Danyang Zhuo, Hao Zhang, Dawn Song, and Ion Stoica. 2021. Terapipe: Token-level pipeline parallelism for training large-scale language models. In *International Conference on Machine Learning*. 6543–6552.
- [27] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889* (2023).
- [28] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172* (2023).
- [29] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. 2018. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162* (2018).
- [30] Maxim Milakov and Natalia Gimelshein. 2018. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867* (2018).
- [31] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.
- [32] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-efficient pipeline-parallel dnn training. In *International Conference on Machine Learning*. PMLR, 7937–7947.
- [33] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [35] Shishir G. Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph E. Gonzalez. 2022. POET: Training Neural Networks on Tiny Devices with Integrated Rematerialization and Paging. In *Proceedings of the 39th International Conference on Machine Learning*, Vol. 162. 17573–17583.
- [36] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5 (2023), 606–624.
- [37] Ofir Press and Lior Wolf. 2016. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859* (2016).
- [38] Penghui Qi, Xinyi Wan, Nyamdavaa Amar, and Min Lin. 2024. Pipeline Parallelism with Controllable Memory. *arXiv:2405.15362 [cs.LG]* <https://arxiv.org/abs/2405.15362>
- [39] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. 2024. Zero Bubble (Almost) Pipeline Parallelism. In *The Twelfth International Conference on Learning Representations*.
- [40] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.
- [41] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [42] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 3505–3506.
- [43] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W Keckler. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–13.
- [44] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1986. Learning representations by back-propagating errors. *nature* 323, 6088 (1986), 533–536.
- [45] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [46] SB Shriram, Anshuj Garg, and Purushottam Kulkarni. 2019. Dynamic memory management for GPU-based training of deep neural networks. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 200–209.
- [47] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David Silver, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy Lillicrap, Angeliki Lazaridou, Orhan Firat, James Molloy, Michael Isard, Paul R. Barham, Tom Hennigan, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, Ryan Doherty, Eli Collins, Clemens Meyer, Eliza Rutherford, Erica Moreira, Kareem Ayoub, Megha Goel, Jack Krawczyk, Cosmo Du, Ed Chi, Heng-Tze Cheng, Eric Ni, Purvi Shah, Patrick Kane, Betty Chan, Manaal Faruqi, Aliaksei Severyn, Hanzhao Lin, YuGuang Li, Yong Cheng, Abe Ittycheriah, Mahdis Mahdieh, Mia Chen, Pei Sun, Dustin Tran, Sumit Bagri, Balaji Lakshminarayanan, Jeremiah Liu, Andras Orban, Fabian Gũra, Hao Zhou, Xinying Song, Aurelien Boffy, Harish Ganapathy, Steven Zheng, HyunJeong Choe, Ȧgoston Weisz, Tao Zhu, Yifeng Lu, Siddharth Gopal, Jarrod Kahn, Maciej Kula, Jeff Pitman, Rushin Shah, Emanuel Taropa, Majd Al Meray, Martin Baeuml, Zhifeng Chen, Laurent El Shafey, Yuying Zhang, Olcan Sercinoglu, George Tucker, Enrique Piqueras, Maxim Krikun, Iain Barr, Nikolay Savinov, Ivo Danihelka, Becca Roelofs, Anais White, Anders Andreassen, Tamara von Glehn, Lakshman Yagati, Mehran Kazemi, Lucas Gonzalez, Misha Khalman, Jakub Sygnowski, Alexandre Frechette, Charlotte Smith, Laura Culp, Lev Proleev, Yi Luan, Xi Chen, James Lottes, Nathan Schucher, Federico Lebron, Alban Rustemi, Natalie Clay, Phil Crone, Tomas Kocisky, Jeffrey Zhao, Bartek Perz, Dian Yu, Heidi Howard, Adam Bloniarz, Jack W. Rae, Han Lu, Laurent Sifre, Marcello Maggioni, Fred Alcober, Dan Garrette, Megan Barnes, Shantanu Thakoor, Jacob Austin, Gabriel Barth-Marón, William Wong, Rishabh Joshi, Rahma Chaabouni, Deeni Fatiha, Arun Ahuja, Gaurav Singh Tomar, Evan Senter, Martin Chadwick, Ilya Kornakov, Nithya Attaluri, İnakı Iturrate, Ruibo Liu, Yunxuan Li, Sarah Cogan, Jeremy Chen, Chao Jia, Chenjie Gu, Qiao Zhang, Jordan Grimstad, Ale Jakse Hartman, Xavier Garcia, Thanumalayan Sankaranarayanan Pillai, Jacob Devlin, Michael Laskin, Diego de Las Casas, Dasha Valter, Connie Tao, Lorenzo Blanco, Adrià Puigdomènech Badia, David Reitter, Mianna Chen, Jenny Brennan, Clara Rivera, Sergey Brin, Shariq Iqbal, Gabriela Surita, Jane Labanowski, Abhi Rao, Stephanie Winkler, Emilio Parisotto, Yiming Gu, Kate Olszewska, Ravi Addanki, Antoine Miech, Annie Louis, Denis Teplyashin, Geoff Brown, Elliot Catt, Jan Bala-guer, Jackie Xiang, Pidong Wang, Zoe Ashwood, Anton Briukhov, Albert Webson, Sanjay Ganapathy, Smit Sanghavi, Ajay Kannan, Ming-Wei Chang, Axel Stjern-gren, Josip Djolonga, Yuting Sun, Ankur Bapna, Matthew Aitchison, Pedram Pejman, Henryk Michalewski, Tianhe Yu, Cindy Wang, Juliette Love, Junwhan Ahn, Dawn Bloxwich, Kehang Han, Peter Humphreys, Thibault Sellam, James Bradbury, Varun Godbole, Sina Samangooei, Bogdan Moc, Alex Kaskasoli, Sébastien M. R. Arnold, Vijay Vasudevan, Shubham Agrawal, Jason Riesa, Dmitry

Lepikhin, Richard Tanburn, Srivatsan Srinivasan, Hyeontaek Lim, Sarah Hodgkinson, Pranav Shyam, Johan Ferret, Steven Hand, Ankush Garg, Tom Le Paine, Jian Li, Yujia Li, Minh Giang, Alexander Neitz, Zaheer Abbas, Sarah York, Machel Reid, Elizabeth Cole, Aakanksha Chowdhery, Dipanjan Das, Dominika Rogoz-ńska, Vitaliy Nikolaev, Pablo Sprechmann, Zachary Nado, Lukas Zilka, Flavien Prost, Luheng He, Marianne Monteiro, Gaurav Mishra, Chris Welty, Josh Newlan, Dawei Jia, Miltiadis Allamanis, Clara Huiyi Hu, Raoul de Liedekerke, Justin Gilmer, Carl Srouf, Shruti Rijhwani, Shaobo Hou, Disha Shrivastava, Anirudh Baddepudi, Alex Goldin, Adnan Ozturk, Albin Cassirer, Yunhan Xu, Daniel Sohn, Devendra Sachan, Reinald Kim Amplayo, Craig Swanson, Dessie Petrova, Shashi Narayan, Arthur Guez, Siddhartha Brahma, Jessica Landon, Miteyan Patel, Ruizhe Zhao, Kevin Vellela, Luyu Wang, Wenhao Jia, Matthew Rahtz, Mai Giménez, Legg Yeung, James Keeling, Petko Georgiev, Diana Mincu, Boxi Wu, Salem Haykal, Rachel Saputo, Kiran Vodrahalli, James Qin, Zeynep Cankara, Abhanshu Sharma, Nick Fernando, Will Hawkins, Behnam Neyshabur, Solomon Kim, Adrian Hutter, Priyanka Agrawal, Alex Castro-Ros, George van den Driessche, Tao Wang, Fan Yang, Shuo yiin Chang, Paul Komarek, Ross McIlroy, Mario Lučić, Guodong Zhang, Wael Farhan, Michael Sharman, Paul Natsev, Paul Hsieh, Yamini Bansal, Siyuan Qiao, Kris Cao, Siamak Shakeri, Christina Butterfield, Justin Chung, Paul Kishan Rubenstein, Shivani Agrawal, Arthur Mensch, Kedar Soparkar, Karel Lenc, Timothy Zhang, Aedan Pope, Loren Maggiore, Jackie Kay, Priya Jhakra, Shibo Wang, Joshua Maynez, Mary Phuong, Taylor Tobin, Andrea Tacchetti, Maja Trebacz, Kevin Robinson, Yash Katariya, Sebastian Riedel, Paige Bailey, Kefan Xiao, Nimesh Ghelani, Lora Aroyo, Ambrose Slone, Neil Houlsby, Xuehan Xiong, Zhen Yang, Elena Gribovskaya, Jonas Adler, Mateo Wirth, Lisa Lee, Music Li, Thais Kagohara, Jay Pavagadhi, Sophie Bridgers, Anna Bortsova, Sanjay Ghemawat, Zafarali Ahmed, Tianqi Liu, Richard Powell, Vijay Bolina, Mariko Iinuma, Polina Zablotskaia, James Besley, Da-Woon Chung, Timothy Dozat, Ramona Comanescu, Xiance Si, Jeremy Greer, Guolong Xu, Martin Polacek, Raphaël Lopez Kaufman, Simon Tokumine, Hexiang Hu, Elena Buchatskaya, Yingjie Miao, Mohamed Elhawaty, Aditya Siddhant, Nenad Tomasev, Jinwei Xing, Christina Greer, Helen Miller, Shereen Ashraf, Aurko Roy, Zizhao Zhang, Ada Ma, Angelos Filos, Milos Besta, Rory Blevins, Ted Klimenko, Chih-Kuan Yeh, Soravit Changpinyo, Jiaqi Mu, Oscar Chang, Mantas Pajarskas, Carrie Muir, Vered Cohen, Charline Le Lan, Krishna Haridasan, Amit Marathe, Steven Hansen, Sholto Douglas, Rajkumar Samuel, Mingqiu Wang, Sophia Austin, Chang Lan, Jiepu Jiang, Justin Chiu, Jaime Alonso Lorenzo, Lars Lowe Sjösund, Sébastien Cevey, Zach Gleicher, Thi Avrahami, Anudhyan Boral, Hansa Srinivasan, Vittorio Selo, Rhys May, Konstantinos Aisopos, Léonard Hussenot, Livio Baldini Soares, Kate Baumli, Michael B. Chang, Adrià Recasens, Ben Caine, Alexander Pritzel, Filip Pavetic, Fabio Pardo, Anita Gergely, Justin Frye, Vinay Ramasesh, Dan Horgan, Kartikeya Badola, Nora Kassner, Subhrajit Roy, Ethan Dyer, Victor Campos Campos, Alex Tomala, Yunhao Tang, Dalia El Badawy, Elspeth White, Basil Mustafa, Oran Lang, Abhishek Jindal, Sharad Vikram, Zhitao Gong, Sergi Caelles, Ross Hemsley, Gregory Thornton, Fangxiaoyu Feng, Wojciech Stokowiec, Ce Zheng, Phoebe Thacker, Çağlar Ünlü, Zhishuai Zhang, Mohammad Saleh, James Svensson, Max Bileschi, Piyush Patil, Ankesh Anand, Roman Ring, Katerina Tsihla, Arpi Vezer, Marco Selvi, Toby Shevlane, Mikael Rodriguez, Tom Kwiatkowski, Samira Daruki, Keran Rong, Allan Dafoe, Nicholas FitzGerald, Keren Gu-Lemberg, Mina Khan, Lisa Anne Hendricks, Marie Pellat, Vladimir Feinberg, James Cobon-Kerr, Tara Sainath, Maribeth Rauh, Sayed Hadi Hashemi, Richard Ives, Yana Hasson, Eric Noland, Yuan Cao, Nathan Byrd, Le Hou, Qingze Wang, Thibault Sottiaux, Michela Paganini, Jean-Baptiste Lespiau, Alexandre Moufarek, Samer Hassan, Kaushik Shivakumar, Joost van Amersfoort, Amol Mandhane, Pratik Joshi, Anirudh Goyal, Matthew Tung, Andrew Brock, Hannah Sheahan, Vedant Misra, Cheng Li, Nemanja Rakićević, Mostafa Dehghani, Fanguy Liu, Sid Mittal, Junhyuk Oh, Seb Noury, Eren Sezener, Fantine Huot, Matthew Lamm, Nicola De Cao, Charlie Chen, Sidharth Mudgal, Romina Stella, Kevin Brooks, Gautam Vasudevan, Chenxi Liu, Mainak Chaitin, Nivedita Melinkeri, Aaron Cohen, Venus Wang, Kristie Seymore, Sergey Zubkov, Rahul Goel, Summer Yue, Sai Krishnakumaran, Brian Albert, Nate Hurley, Motoki Sano, Anhad Mohanbey, Jonah Joughin, Egor Filonov, Tomasz Kepa, Yomna Eldawy, Jiawen Lim, Rahul Rishi, Shirin Badiezedegan, Taylor Bos, Jerry Chang, Sanil Jain, Sri Gayatri Sundara Padmanabhan, Subha Puttagunta, Kalpesh Krishna, Leslie Baker, Norbert Kalb, Vamsi Bedapudi, Adam Kurzkro, Shuntong Lei, Anthony Yu, Oren Litvin, Xiang Zhou, Zhichun Wu, Sam Sobell, Andrea Sciliano, Alan Papir, Robby Neale, Jonas Bragagnolo, Tej Toor, Tina Chen, Valentin Anklin, Feiran Wang, Richie Feng, Milad Gholami, Kevin Ling, Lijuan Liu, Jules Walter, Hamid Moghaddam, Arun Kishore, Jakub Adamek, Tyler Mercado, Jonathan Mallinson, Siddhinita Wandekar, Stephen Cagle, Eran Ofek, Guillermo Garrido, Clemens Lombriser, Maksim Mukha, Botu Sun, Hafeezul Rahman Mohammad, Josip Matak, Yadi Qian, Vikas Peswani, Pawel Janus, Quan Yuan, Leif Schellin, Oana David, Ankur Garg, Yifan He, Oleksii Duzhyi, Anton Aligmyr, Timothée Lottaz, Qi Li, Vikas Yadav, Luyao Xu, Alex Chinien, Rakesh Shivanna, Aleksandr Chuklin, Josie Li, Carrie Spadine, Travis Wolfe, Kareem Mohamed, Subhabrata Das, Zihang Dai, Kyle He, Daniel von Dincklage, Shyam Upadhyay, Akanksha Maurya, Luyan Chi, Sebastian Krause, Khalid Salama, Pam G Rabinovitch, Pavan Kumar Reddy M, Aarush Selvan, Mikhail Dekhtyarev, Golnaz Ghiasi, Erdem Guven,

Himanshu Gupta, Boyi Liu, Deepak Sharma, Idan Heimlich Shtacher, Shachi Paul, Oscar Akerlund, François-Xavier Aubet, Terry Huang, Chen Zhu, Eric Zhu, Elico Teixeira, Matthew Fritze, Francesco Bertolini, Liana-Eleonora Marinescu, Martin Bølle, Dominik Paulus, Khyatti Gupta, Tejasj Latkar, Max Chang, Jason Sanders, Roopa Wilson, Xuewei Wu, Yi-Xuan Tan, Lam Nguyen Thiet, Tulsee Doshi, Sid Lal, Swaroop Mishra, Wanming Chen, Thang Luong, Seth Benjamin, Jasmine Lee, Ewa Andrejczuk, Dominik Rabiej, Vipul Ranjan, Krzysztof Styrz, Pengcheng Yin, Jon Simon, Malcolm Rose Harriott, Mudit Bansal, Alexei Robsky, Geoff Bacon, David Greene, Daniil Mirylenka, Chen Zhou, Obaid Sarvana, Abhimanyu Goyal, Samuel Andermatt, Patrick Siegler, Ben Horn, Assaf Israel, Francesco Pongetti, Chih-Wei "Louis" Chen, Marco Selvatici, Pedro Silva, Kathie Wang, Jackson Tolins, Kelvin Guu, Roey Yogev, Xiaochen Cai, Alessandro Agostini, Maulik Shah, Hung Nguyen, Noah O'Donnaile, Sébastien Pereira, Linda Friso, Adam Stambler, Adam Kurzkro, Chenkai Kuang, Yan Romanikhin, Mark Geller, ZJ Yan, Kane Jang, Cheng-Chun Lee, Wojciech Fica, Eric Malmi, Qijun Tan, Dan Bania, Daniel Balle, Ryan Pham, Yanping Huang, Diana Avram, Hongzhi Shi, Jasjit Singh, Chris Hideo, Niharika Ahuja, Pranab Saxena, Dan Dooley, Srividya Pranavi Potharaju, Eileen O'Neill, Anand Gokulchandran, Ryan Foley, Kai Zhao, Mike Dusenberry, Yuan Liu, Pulkit Mehta, Ragha Kotikalapudi, Chalence Safranek-Shrader, Andrew Goodman, Joshua Kessinger, Eran Globen, Prateek Kolhar, Chris Gorgolewski, Ali Ibrahim, Yang Song, Ali Eichenbaum, Thomas Brovelli, Sahitya Potluri, Preethi Lahoti, Cip Baetu, Ali Ghorbani, Charles Chen, Andy Crawford, Shalini Pal, Mukund Sridhar, Petru Gurita, Asier Mujika, Igor Petrovski, Pierre-Louis Cedoz, Chenmei Li, Shiyuan Chen, Niccolò Dal Santo, Siddharth Goyal, Jitesh Punjabi, Karthik Kappaganthu, Chester Kwak, Pallavi LV, Sarmishta Velury, Himadri Choudhury, Jamie Hall, Premal Shah, Ricardo Figueira, Matt Thomas, Minjie Lu, Ting Zhou, Chintu Kumar, Thomas Jurdi, Sharat Chikkerur, Yenai Ma, Adams Yu, Soo Kwak, Victor Åhdel, Sujevan Rajayogam, Travis Choma, Fei Liu, Aditya Barua, Colin Ji, Ji Ho Park, Vincent Hellendorn, Alex Bailey, Taylan Bilal, Huanjie Zhou, Mehrdad Khatir, Charles Sutton, Wojciech Rzdowski, Fiona Macintosh, Konstantin Shagin, Paul Medina, Chen Liang, Jinjing Zhou, Pararth Shah, Yingying Bi, Attila Dankovics, Shipra Banga, Sabine Lehmann, Marissa Bredezen, Zifan Lin, John Eric Hoffmann, Jonathan Lai, Raynald Chung, Kai Yang, Nihal Balani, Arthur Bražinskas, Andrei Sozanschi, Matthew Hayes, Héctor Fernández Alcalde, Peter Makarov, Will Chen, Antonio Stella, Liselotte Snijders, Michael Mandl, Ante Kärrman, Pawel Nowak, Xinyi Wu, Alex Dyck, Krishnan Vaidyanathan, Raghavender R, Jessica Mallet, Mitch Rudominer, Eric Johnston, Sushil Mittal, Akhil Udathu, Janara Christensen, Vishal Verma, Zach Irving, Andreas Santucci, Gamaleldin Elsayed, Elnaz Davoudi, Marin Georgiev, Ian Tenney, Nan Hua, Geoffrey Cideron, Edouard Leurent, Mahmoud Alnahlawi, Ionut Georgescu, Nan Wei, Ivy Zheng, Dylan Scandinaro, Heinrich Jiang, Jasper Snoek, Mukund Sundararajan, Xuezhi Wang, Zack Ontiveros, Itay Karo, Jeremy Cole, Vinu Rajashekhar, Lara Tumeh, Eyal Ben-David, Rishub Jain, Jonathan Uesato, Romina Datta, Oskar Bunyan, Shimu Wu, John Zhang, Piotr Stanczyk, Ye Zhang, David Steiner, Subhajit Naskar, Michael Azzam, Matthew Johnson, Adam Paszke, Chung-Cheng Chiu, Jaume Sanchez Elias, Afroz Mohiuddin, Faizan Muhammad, Jin Miao, Andrew Lee, Nino Vieillard, Jane Park, Jiageng Zhang, Jeff Stanway, Drew Garmon, Abhijit Karmarkar, Zhe Dong, Jong Lee, Aviral Kumar, Luowei Zhou, Jonathan Evens, William Isaac, Geoffrey Irving, Edward Loper, Michael Fink, Isha Arkatkar, Nanxin Chen, Izhak Shafiran, Ivan Petychenko, Zhe Chen, Johnson Jia, Anselm Levskaya, Zhenkai Zhu, Peter Grabowski, Yu Mao, Alberto Magni, Kaisheng Yao, Javier Snaider, Norman Casagrande, Evan Palmer, Paul Suganthan, Alfonso Castaño, Irene Giannoumis, Wooyeol Kim, Mikolaj Rybiński, Ashwin Sreevatsa, Jennifer Prendki, David Soergel, Adrian Goedeckemeyer, Willi Gierke, Mohsen Jafari, Meenu Gaba, Jeremy Wiesner, Diana Gage Wright, Yawen Wei, Harsha Vashisht, Yana Kulizhskaya, Jay Hoover, Maigo Le, Lu Li, Chimezie Iwuanyanwu, Lu Liu, Kevin Ramirez, Andrey Khorlin, Albert Cui, Tian LIN, Marcus Wu, Ricardo Aguilar, Keith Pallo, Abhishek Chakladar, Ginger Perng, Elena Allica Abellan, Mingyang Zhang, Ishita Dasgupta, Nate Kushman, Ivo Penchev, Alena Repina, Xihui Wu, Tom van der Weide, Priya Ponnappalli, Caroline Kaplan, Jiri Simsa, Shuangfeng Li, Olivier Dousse, Fan Yang, Jeff Piper, Nathan Ie, Rama Pasumarthi, Nathan Lintz, Anitha Vijayakumar, Daniel Andor, Pedro Valenzuela, Minnie Lui, Cosmin Paduraru, Daiyi Peng, Katherine Lee, Shuyuan Zhang, Somer Greene, Duc Dung Nguyen, Paula Kurylowicz, Cassidy Hardin, Lucas Dixon, Lili Janzer, Kiam Choo, Ziqiang Feng, Biao Zhang, Achintya Singhal, Dayou Du, Dan McKinnon, Natasha Antropova, Tolga Bolukbasi, Orgad Keller, David Reid, Daniel Finchelstein, Maria Abi Raad, Remi Crocker, Peter Hawkins, Robert Dadashi, Colin Gaffney, Ken Franko, Anna Bulanova, Rémi Leblond, Shirley Chung, Harry Askham, Luis C. Cobo, Kelvin Xu, Felix Fischer, Jun Xu, Christina Sorokin, Chris Alberti, Chu-Cheng Lin, Colin Evans, Alek Dimitriev, Hannah Forbes, Dylan Banarse, Zora Tung, Mark Omernick, Colton Bishop, Rachel Sterneck, Rohan Jain, Jiawei Xia, Ehsan Amid, Francesco Piccinno, Xingyu Wang, Praseem Banzal, Daniel J. Mankowitz, Alex Polozov, Victoria Krakovna, Sasha Brown, MohammadHossein Bateni, Dennis Duran, Vlad Firoiu, Meghana Thotakuri, Tom Natan, Matthieu Geist, Ser tan Gigin, Hui Li, Jiayu Ye, Ofir Roval, Reiko Tojo, Michael Kwong, James Lee-Thorp, Christopher Yew, Danila Sinopalnikov, Sabela Ramos, John Mellor, Abhishek Sharma, Kathy Wu,

- David Miller, Nicolas Sonnerat, Denis Vnukov, Rory Greig, Jennifer Beattie, Emily Caveness, Libin Bai, Julian Eisenschlos, Alex Korchemniy, Tomy Tsai, Mimi Jasarevic, Weize Kong, Phuong Dao, Zeyu Zheng, Frederick Liu, Fan Yang, Rui Zhu, Tian Huey Teh, Jason Sammiya, Evgeny Gladchenko, Nejc Trdin, Daniel Toyama, Evan Rosen, Sasan Tavakkol, Linting Xue, Chen Elkind, Oliver Woodman, John Carpenter, George Papamakarios, Rupert Kemp, Sushant Kafle, Tanya Grunina, Rishika Sinha, Alice Talbert, Diane Wu, Denese Owusu-Afriyie, Cosmo Du, Chloe Thornton, Jordi Pont-Tuset, Pradyumna Narayana, Jing Li, Saaber Fatehi, John Wieting, Omar Ajmeri, Benigno Uria, Yeongil Ko, Laura Knight, Amélie Héliou, Ning Niu, Shane Gu, Chenxi Pang, Yeqing Li, Nir Levine, Ariel Stolovich, Rebecca Santamaria-Fernandez, Sonam Goenka, Wenny Yustalim, Robin Strudel, Ali Elqursh, Charlie Deck, Hyo Lee, Zonglin Li, Kyle Levin, Raphael Hoffmann, Dan Holtmann-Rice, Olivier Bachem, Sho Arora, Christy Koh, Soheil Hassas Yeganeh, Siim Pöder, Mukarram Tariq, Yanhua Sun, Lucian Ionita, Mojtaba Seyedhosseini, Pouya Tafti, Zhiyu Liu, Anmol Gulati, Jasmine Liu, Xinyu Ye, Bart Chrzascz, Lily Wang, Nikhil Sethi, Tianrun Li, Ben Brown, Shreya Singh, Wei Fan, Aaron Parisi, Joe Stanton, Vinod Koverkathu, Christopher A. Choquette-Choo, Yunjie Li, TJ Lu, Abe Ittycheriah, Prakash Shroff, Mani Varadarajan, Sanaz Bahargam, Rob Willoughby, David Gaddy, Guillaume Desjardins, Marco Cornero, Brona Robenek, Bhavishya Mittal, Ben Albrecht, Ashish Shenoy, Fedor Moiseev, Henrik Jacobsson, Alireza Ghaffarkhah, Morgane Rivière, Alanna Walton, Clément Crepy, Alicia Parrish, Zongwei Zhou, Clement Farabet, Carey Radebaugh, Praveen Srinivasan, Claudia van der Salm, Andreas Fidlend, Salvatore Scellato, Eri Latorre-Chimoto, Hanna Klimczak-Plucińska, David Bridson, Dario de Cesare, Tom Hudson, Piermaria Mendolicchio, Lexi Walker, Alex Morris, Matthew Mauger, Alexey Guseynov, Alison Reid, Seth Odoom, Lucia Lohar, Victor Cotrutu, Madhavi Venugula, Dominik Grewe, Anastasia Petrushkina, Tom Duerig, Antonio Sanchez, Steve Yadlowsky, Amy Shen, Amir Globerson, Lynette Webb, Sahil Dua, Dong Li, Surya Bhupatiraju, Dan Hurt, Haroon Qureshi, Ananth Agarwal, Tomer Shani, Matan Eyal, Anuj Khare, Shreyas Rammohan Belle, Lei Wang, Chetan Tekur, Mihir Sanjay Kale, Jinliang Wei, Ruoxin Sang, Brennan Saeta, Tyler Liechty, Yi Sun, Yao Zhao, Stephan Lee, Pandu Nayak, Doug Fritz, Manish Reddy Vuyyuru, John Aslanides, Nidhi Vyas, Martin Wicke, Xiao Ma, Evgenii Eltyshv, Nina Martin, Hardie Cate, James Manyika, Keyvan Amiri, Yelin Kim, Xi Xiong, Kai Kang, Florian Luisier, Nilesch Tripuraneni, David Madras, Mandy Guo, Austin Waters, Oliver Wang, Joshua Ainslie, Jason Baldridge, Han Zhang, Garima Pruthi, Jakob Bauer, Feng Yang, Riham Mansour, Jason Gelman, Yang Xu, George Polovets, Ji Liu, Honglong Cai, Warren Chen, Xiang-Hai Sheng, Emily Xue, Sherjil Ozair, Christof Angermueller, Xiaowei Li, Anoop Sinha, Weiren Wang, Julia Wiesinger, Emmanouil Koukoumidis, Yuan Tian, Anand Iyer, Madhu Gurumurthy, Mark Goldenson, Parashar Shah, MK Blake, Hongkun Yu, Anthony Urbanowicz, Jennimaria Palomaki, Chrisantha Fernando, Ken Durden, Harsh Mehta, Nikola Momchev, Elahe Rahimtoroghi, Maria Georgaki, Amit Raul, Sebastian Ruder, Morgan Redshaw, Jinhyuk Lee, Denny Zhou, Komal Jalan, Dinghua Li, Blake Hechtman, Parker Schuh, Milad Nasr, Kieran Milan, Vladimir Mikulic, Juliana Franco, Tim Green, Nam Nguyen, Joe Kelley, Aroma Mahendru, Andrea Hu, Joshua Howland, Ben Vargas, Jeffrey Hui, Kshitij Bansal, Vikram Rao, Rakesh Ghiya, Emma Wang, Ke Ye, Jean Michel Sarr, Melanie Moranski Preston, Madeleine Elish, Steve Li, Aakash Kaku, Jigar Gupta, Ice Pasupat, Da-Cheng Juan, Milan Someswar, Teji M., Xinyun Chen, Aida Amini, Alex Fabrikant, Eric Chu, Xuanyi Dong, Amruta Muthal, Senaka Buthpitiya, Sarthak Jauhari, Nan Hua, Urvashi Khandelwal, Ayal Hitron, Jie Ren, Larissa Rinaldi, Shahar Drath, Avigail Dabush, Nan-Jiang Jiang, Harshal Godhia, Uli Sachs, Anthony Chen, Yicheng Fan, Hagai Taitelbaum, Hila Noga, Zhuyun Dai, James Wang, Chen Liang, Jenny Hamer, Chun-Sung Ferng, Chenel Elkind, Aviel Atlas, Paulina Lee, Vit Listik, Mathias Carlen, Jan van de Kerkhof, Marcin Pikuś, Krunoslav Zaher, Paul Müller, Sasha Zykova, Richard Stefanec, Vitaly Gatsko, Christoph Hirsenschall, Ashwin Sethi, Xingyu Federico Xu, Chetan Ahuja, Beth Tsai, Anca Stefanioiu, Bo Feng, Keshav Dhandhanian, Manish Katyal, Akshay Gupta, Atharva Parulekar, Divya Pitta, Jing Zhao, Vivaan Bhatia, Yashodha Bhavnani, Omar Alhadlaq, Xiaolin Li, Peter Danenberg, Dennis Tu, Alex Pine, Vera Filippova, Abhipso Ghosh, Ben Limonchik, Bhargava Urala, Chaitanya Krishna Lanka, Derik Clive, Yi Sun, Edward Li, Hao Wu, Kevin Hongtongsak, Ianna Li, Kalind Thakkar, Kuanysh Omarov, Kushal Majmudar, Michael Alverson, Michael Kucharski, Mohak Patel, Mudit Jain, Maksim Zabelin, Paolo Pelagatti, Rohan Kohli, Saurabh Kumar, Joseph Kim, Swetha Sankar, Vineet Shah, Lakshmi Ramachandruni, Xiangkai Zeng, Ben Bariach, Laura Weidinger, Tu Vu, Alek Andreev, Antoine He, Kevin Hui, Sheleem Kashem, Amar Subramanya, Sissie Hsiao, Demis Hassabis, Koray Kavukcuoglu, Adam Sadovsky, Quoc Le, Trevor Strohmman, Yonghui Wu, Slav Petrov, Jeffrey Dean, and Oriol Vinyals. 2024. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805 [cs.CL] <https://arxiv.org/abs/2312.11805>
- [48] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* (2024).
- [49] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [50] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [51] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrutie Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madsen Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [53] Eric P. Xing, Qirong Ho, Wei Dai, Jin Kyu Kim, Jinliang Wei, Seunghak Lee, Xun Zheng, Pengtao Xie, Abhimanu Kumar, and Yaoliang Yu. 2015. Petuum: A New Platform for Distributed Machine Learning on Big Data. *IEEE Transactions on Big Data* 1, 2 (2015), 49–67. doi:10.1109/TBDATA.2015.2472014
- [54] Tailing Yuan, Yuliang Liu, Xucheng Ye, Shenglong Zhang, Jianchao Tan, Bin Chen, Chengru Song, and Di Zhang. 2024. Accelerating the Training of Large Language Models using Efficient Activation Rematerialization and Optimal Hybrid Parallelism. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 545–561.
- [55] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906* (2022).

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

A Overview of Contributions and Artifacts

A.1 Paper’s Main Contributions

In summary, our paper presents the following contributions:

- C₁ Our analysis reveals two fundamental limitations in current pipeline parallelism approaches: memory inefficiency and significant pipeline bubble overhead. Moreover, we demonstrate that these issues exacerbate their difficulties in long-context training.
- C₂ We propose SlimPipe, a novel approach to fine-grained pipeline parallelism that employs uniform sequence slicing coupled with 1F1B scheduling.
- C₃ We propose a novel workload redistribution technique. By dynamically reallocating the attention workload between pipeline devices handling different slices, we achieve a balanced distribution of both computational and memory load among them.
- C₄ Our extensive evaluation demonstrates that SlimPipe consistently outperforms state-of-the-art systems, such as DeepSpeed and Megatron-LM, achieving significant improvements in both memory utilization and throughput.

A.2 Computational Artifacts

The source code of our system (SlimPipe) and the scripts for evaluation are archived at <https://doi.org/10.5281/zenodo.15917750>, and the code will be open-sourced later. As for other source codes (e.g. those of the compared pipeline schemes), we download them from their repository, respectively. URLs of these repositories are listed in Table 5

Artifact ID	Contributions Supported	Related Paper Elements
A ₁	C ₂	Figure 10
A ₂	C ₂ , C ₃	Figure 11
A ₃	C ₄	Figure 12
A ₄	C ₄	Table 4
A ₅	C ₁ , C ₂ , C ₃	Figures 13-14

B Artifact Identification

B.1 Computational Artifact A₁

Relation To Contributions

This artifact demonstrates that the model states and activations are both virtually distributed in pipeline parallelism. In another word, SlimPipe is memory-thrifty.

Expected Results

The memory consumptions measured on the first and the last devices will be both close to the theoretically calculated one. And those values will be in an inverse proportional relationship to the pipeline parallelism size.

Expected Reproduction Time (in Minutes)

The setup time is about 10 minutes. The expected execution time is about 30 minutes. And the analysis costs a few seconds.

Artifact Setup (incl. Inputs)

Hardware. We conduct experiments on a GPU cluster. Each cluster node is equipped with two Intel Xeon Platinum CPUs, 1TB of memory, and 8 NVIDIA H800 80GB GPUs. The GPUs within a node are interconnected via NVLink, whose bandwidth is 400GB/s per GPU. Besides that, every GPU is coupled with a 400Gbps NIC for inter-node communication.

Software. The software required to build our system is listed in Table 6.

Datasets / Inputs. We use a subset of the openwebtext dataset (<https://huggingface.co/datasets/stanford-cs336/owt-sample>). We use the tokenizer from Llama3 (<https://huggingface.co/meta-llama/Llama-3.2-1B>). Actually our system can also be used with other datasets or tokenizers.

Installation and Deployment. The same GPU cluster, software packages, dataset and tokenizer are used for following artifacts.

Artifact Execution

We use the following configurations for this experiment.

- (1) The model being used is Llama 13B.
- (2) The sequence lengths used are 32 K, 64 K and 96 K. The global batch size is 2 and the micro-batch size is 1.
- (3) Tensor parallelism size is set to 8 ($t = 8$).
- (4) Pipeline parallelism size increases from 2 to 8 ($p \in \{2, 4, 8\}$). The number of layers per virtual pipeline stage is set to 1 ($v = L/p$).
- (5) The number of slices per sequence is four times the pipeline parallelism size ($n = 4p$).
- (6) For a specific configuration, 2 iterations are executed, the memory footprint reported during the first iteration is discarded.

We enumerate the PP sizes and sequence lengths, train the model with that specifications and record the memory usage on each node. We plot the memory usage numbers as dots.

Artifact Analysis (incl. Outputs)

The outputs are a set of logs. We parse each log to get the memory usage number.

B.2 Computational Artifact A₂

Relation To Contributions

This artifact helps us understand how to choose an appropriate number of slices in SlimPipe. And SlimPipe with a moderate number of slices can improve the training efficiency while saving memory.

Table 5: Source codes of compared pipeline schemes.

Pipeline Scheme	URL
Default 1F1B	https://github.com/NVIDIA/Megatron-LM/tree/core_r0.8.0
Interleaved 1F1B	https://github.com/NVIDIA/Megatron-LM/tree/core_r0.8.0
ZB-V	https://github.com/sail-sg/zero-bubble-pipeline-parallelism/tree/zero-bubble-v0.1.0
V-Half	https://github.com/sail-sg/zero-bubble-pipeline-parallelism/tree/zero-bubble-v0.1.0
SlimPipe	https://github.com/kwai/Megatron-Kwai/tree/slimpipe (will be open-sourced later)

Table 6: Softwares used to build our system.

Software	Version	URL
GCC	9.4.0	https://gcc.gnu.org/
Python	3.10.12	https://www.python.org/
CUDA Toolkit	12.6	https://developer.nvidia.com/cuda-toolkit
NCCL	2.21.5	https://github.com/NVIDIA/nccl/
PyTorch	2.6.0	https://pytorch.org/
Apex	0.1	https://github.com/NVIDIA/apex
transformers	4.39.3	https://github.com/huggingface/transformers
cuDNN	9.8.0	https://developer.nvidia.com/cudnn
TransformerEngine	1.12	https://github.com/NVIDIA/TransformerEngine
grouped_gemm	1.1.4	https://github.com/fanshiqing/grouped_gemm

Expected Results

When increasing the number of slices, the MFU will increase at the beginning, and then decrease at the end. The transition point should come later for longer sequence lengths. This indicates that fine-grained slicing can reduce pipeline warmup bubbles, but it will damage the arithmetic intensity when slicing too fine.

Expected Reproduction Time (in Minutes)

The execution time is about 240 minutes.

Artifact Setup (incl. Inputs)

The basic setup is the same as described in Section B.1.

Artifact Execution

We use the following parameters for this experiment.

- (1) The model being used is Llama 13B.
- (2) The sequence lengths used are 128 K, 256 K and 512 K. The global batch size is 2 and the micro-batch size is 1.
- (3) Tensor parallelism size is set to 8 ($t = 8$).
- (4) Pipeline parallelism size is set to 4 ($p = 4$). The number of layers per virtual pipeline stage is set to 2 ($v = L/p/2$).
- (5) Full activation recomputation is employed.
- (6) The number of slices per sequence is equal to the pipeline parallelism size initially, and it's gradually increased to eight times the pipeline parallelism size.
- (7) For a specific configuration, 10 iterations are executed, the statistics reported during the first 5 iterations are discarded.

We enumerate the sequence lengths and slice numbers, train the model with that specifications and record the throughputs.

Artifact Analysis (incl. Outputs)

The outputs are a set of logs recording the TPS numbers per iter. We calculate the average TPS number from each log and convert it to an MFU number.

B.3 Computational Artifact A_3

Relation To Contributions

This artifact compares the end-to-end performance of three systems: DeepSpeed, Megatron-LM and Megatron-LM integrated with SlimPipe. The results show that SlimPipe substantially improves the system capability and efficiency in training long-context LLMs.

Expected Results

The system with SlimPipe will outperform the other two systems in all test cases. DeepSpeed and Megatron-LM will run into out-of-memory error in some test cases. The measured MFUs should be the same as those in Figure 12 in our paper.

Expected Reproduction Time (in Minutes)

The execution time is about 1080 minutes.

Artifact Setup (incl. Inputs)

The basic setup is the same as described in Section B.1. The systems with and without SlimPipe are both from the same source code (modified from NVIDIA Megatron-LM) and are distinguished by system arguments. We adopted DeepSpeed from the source code on March 27, 2025 (git commit ac295aa). We unify the model definitions of these systems with the help of Megatron-DeepSpeed (git commit 3e1da1f)

Artifact Execution

In this experiment, we run each system with the corresponding configurations. We have pre-compiled a list of optimal configurations, which is found by using grid search. The configuration files of SlimPipe and Megatron-LM are listed below.

```
./csv/all-experiments-slimpipe-128.csv
./csv/all-experiments-slimpipe-256.csv
./csv/all-experiments-slimpipe-512.csv
./csv/all-experiments-megatron-128.csv
./csv/all-experiments-megatron-256.csv
./csv/all-experiments-megatron-512.csv
```

A small sample of the CSV files is shown below.

```
#GPUs,s,Model,B,b,t,c,p,e,d,n,v,l,recompute
128,65536,LLaMA-150B-GQA,64,1,8,1,4,1,4,4,24,1,partial+
fc1,61.87
128,131072,LLaMA-150B-GQA,32,1,8,1,8,1,2,8,12,1,partial+
fc1,50.64
128,262144,LLaMA-150B-GQA,16,1,4,2,16,1,1,16,6,1,partial+
fc1,54.12
```

The meaning of each field is explained below.

- #GPUs: number of GPUs to use.
- s: sequence length
- Model: model to use
- B: global batch size
- b: micro-batch size
- t: tensor parallel size
- c: context parallel size
- p: pipeline parallel size
- e: expert parallel size
- d: data parallel size
- n: number of slices per sequence
- v: number of stages per pipeline device
- l: number of layers per virtual stage
- recompute: recomputation strategy, valid options include no (no recomputation), partial+fc1 (selective recomputation), full (full recomputation)

Artifact Analysis (incl. Outputs)

The experiment produces a bunch of execution logs.

B.4 Computational Artifact A_4

Relation To Contributions

This artifact shows the maximum context length that our system can accommodate by using 256 GPUs. And its training efficiency is still high when training with extremely long context lengths.

Expected Results

With the given hybrid parallelism configurations, our system can train Llama 70B with a context length of 2048K, Llama 149B with 1024K, Mixtral 8x7B with 4096K, and Mixtral 8x22B with 2048K. Their training efficiencies will be above 40%.

Expected Reproduction Time (in Minutes)

The execution time is about 300 minutes.

Artifact Setup (incl. Inputs)

The basic setup is the same as described in Section B.1.

Artifact Execution

For this experiment, we have tuned our system and offered pre-compiled configurations to achieve decent performance in ultra-long-context scenarios. The configurations are assembled in the file `./csv/extended_configs.csv`. The meaning of configurations is explained in Section B.3. An additional parameter *offload ratio* is configured to effectively utilize the activation offloading.

Artifact Analysis (incl. Outputs)

The experiment produces a bunch of execution logs.

B.5 Computational Artifact A_5

Relation To Contributions

This artifact compares the training efficiency of different pipeline schemes. This demonstrates that SlimPipe is more memory-thrifty and more efficient than other pipeline schemes.

Expected Results

SlimPipe will run with longer context lengths, while others will run into an out-of-memory error. SlimPipe should deliver MFUs equal to or higher than others across all context lengths. SlimPipe should consume less GPU memory than others across all context lengths.

Expected Reproduction Time (in Minutes)

The execution time is about 60 minutes.

Artifact Setup (incl. Inputs)

The basic setup is the same as described in Section B.1. The compared pipeline schemes and their source codes are list in Table 5. For all pipeline schemes, we use the cuDNN Scaled Dot Product Attention (SDPA) as the implementation of the attention layer.

Artifact Execution

We use the following parameters for this experiment.

- (1) The model being used is Llama 13B.
- (2) The sequence lengths used are 32 K, 64 K, 128 K, 256 K and 512 K. The global batch size is 4 and the micro-batch size is 1.
- (3) Tensor parallelism size is set to 8 ($t = 8$).
- (4) Pipeline parallelism size is set to 4 ($p = 4$). The number of layers per virtual pipeline stage for interleaved 1F1B and SlimPipe is set to 2 ($v = L/p/2$).
- (5) Full activation recomputation is employed for all pipeline parallelism schemes.
- (6) For a specific configuration, 10 iterations are executed, the statistics reported during the first 5 iterations are discarded.

Artifact Analysis (incl. Outputs)

The outputs of the experiment are a bunch of logs.

Artifact Evaluation (AE)

C.1 Computational Artifact A_1

Artifact Setup (incl. Inputs)

- (1) Download and install the software listed in Table 6. Please refer to their installation documents.
- (2) Unpack the artifacts and enter the subdirectory to execute subsequent commands.

```
tar -xvzf slimpipes.tar.gz
cd ./slimpipes/examples/sc25slimpipes
```

- (3) Download the tokenizer. You may first apply for access on the HuggingFace website.

```
git clone https://huggingface.co/meta-llama/Llama-3.2-1B
```

- (4) Download the dataset and preprocess it.

```
./make_dataset.sh
```

Artifact Execution

This experiment can be carried out by executing the following script.

```
./train_all_p_scaling.sh
```

This script first setups the basic configurations (e.g. the model and data specifications, the TP size.). Then it iterates through all the PP sizes and sequence lengths and execute the corresponding trainings.

Artifact Analysis (incl. Outputs)

The outputs of the experiment are a set of execution logs. The logs are organized in folders.

```
slimpipes
+ Llama-13B
+ tp8
+ pp2
| + 2025_04_08_02_25_16-2-96k.txt
| + 2025_04_08_02_32_32-2-64k.txt
| + 2025_04_08_02_39_38-2-32k.txt
+ pp4
| + 2025_04_08_02_27_38-2-96k.txt
| + 2025_04_08_02_34_51-2-64k.txt
| + 2025_04_08_02_41_46-2-32k.txt
+ pp8
+ 2025_04_08_02_30_02-2-96k.txt
+ 2025_04_08_02_37_09-2-64k.txt
+ 2025_04_08_02_44_07-2-32k.txt
```

The peak memory usages are obtained by running the following command.

```
python p_scale_analysis.py --root_dir /path/to/logs/
```

We then use the data to draw a plot showing the relationship between pipeline parallelism sizes and peak memory footprints.

```
cd plots && python p_scaling.py
```

We also obtain theoretical memory consumption for different sequence lengths with the following code.

```
num_params = 13_344_855_040
V = 128000
a = 8
g = 8
h = 5120
H = 13824
L = 40
b = 1
tp = 8
factor = (8 + 4 * g / a + 6 * H / h) * b * h * L

def get_memory(seq_length):
    weight_mem = 18 * num_params
    activation_mem = (
        factor * seq_length
        + 6 * b * seq_length * V
        + 2 * b * seq_length * h
    )
    gpu_mem = weight_mem + activation_mem
    return gpu_mem / tp / 1024**3
```

C.2 Computational Artifact A_2

Artifact Setup (incl. Inputs)

The basic setup is the same as described in Section C.1.

Artifact Execution

This experiment can be carried out by executing the following script.

```
./train_all_n_selection.sh
```

Artifact Analysis (incl. Outputs)

The outputs of the experiment are a set of execution logs. The logs are organized in folders. The Model FLOPs Utilization (MFU) information can be obtained by running the following commands.

```
for p in {1..8}; do python analysis_logs.py --folder-path /path/to/logs/n_selection/128K/pp_l=2/13b/${p} --model-name LLaMA-13B; done
for p in {1..8}; do python analysis_logs.py --folder-path /path/to/logs/n_selection/256K/pp_l=2/13b/${p} --model-name LLaMA-13B; done
for p in {1..8}; do python analysis_logs.py --folder-path /path/to/logs/n_selection/512K/pp_l=2/13b/${p} --model-name LLaMA-13B; done
```

We then draw a line plot showing how number of slices per sequence affect the achieved MFU for different sequence lengths.

```
cd plots && python n_selection.py
```

C.3 Computational Artifact A_3

Artifact Setup (incl. Inputs)

The basic setup is the same as described in Section C.1. Download the DeepSpeed and Megatron-DeepSpeed and patch them with our patches.

```
git clone https://github.com/deepspeedai/DeepSpeed && cd
DeepSpeed && git checkout ac295aa && git apply ../
deepspeed.patch && cd ..
git clone https://github.com/deepspeedai/Megatron-
DeepSpeed && cd Megatron-DeepSpeed && git checkout 3
e1da1f && git apply ../megatron_deepspeed.patch &&
cd ..
```

Artifact Execution

The end-to-end experiment can be reproduced by executing the following commands.

```
# SlimPipe
./train_all_csv.py ./csv/all-experiments-slimpipe-128.csv
./train_all_csv.py ./csv/all-experiments-slimpipe-256.csv
./train_all_csv.py ./csv/all-experiments-slimpipe-512.csv

# Megatron-LM
./train_all_csv.py ./csv/all-experiments-megatron-128.csv
./train_all_csv.py ./csv/all-experiments-megatron-256.csv
./train_all_csv.py ./csv/all-experiments-megatron-512.csv

# DeepSpeed
./train_all_deepspeed.py
```

The script `train_all_csv.py` reads system configurations from a CSV file and runs the configured trainings. The DeepSpeed's configuration is already embedded in its execution script. The meaning of configurations are explained in the corresponding section B.3.

Artifact Analysis (incl. Outputs)

The experiment produces a bunch of execution logs. The log analysis can be carried out with the following command.

```
python analysis_system.py --root_dir /path/to/logs
```

The script will gather relevant information and write results into a CSV file. We then use the data to draw multiple bar plots showing the system performance across different models and sequence lengths.

```
cd plots && python system.py
```

C.4 Computational Artifact A_4

Artifact Setup (incl. Inputs)

The basic setup is the same as described in Section C.1.

Artifact Execution

The experiment can be reproduced by running the following command.

```
./train_all_extended.py ./csv/extended_configs.csv
```

The script `train_all_extended.py` reads system configurations from a CSV file and runs the configured trainings. The meaning of configurations is explained in Section B.3.

Artifact Analysis (incl. Outputs)

We use the following command to get the MFU of individual runs.

```
python calc_mfu_on_log.py --path /path/to/logs
```

Based on the MFU data, we create a table showing the performance of our system for ultra-long-context training.

C.5 Computational Artifact A_5

Artifact Setup (incl. Inputs)

The basic setup is the same as described in Section C.1. Download the source code of compared pipeline schemes in Table 5. Patch the Megatron-LM with `./megatron_core_r0.8.0.patch`. Patch the zero-bubble-pipeline-parallelism with `./zero_bubble_pp.patch`.

Artifact Execution

Artifact Analysis (incl. Outputs)

The logs can be analyzed with the following commands.

```
python analysis_logs.py --folder-path /path/to/logs/1f1b
--model-name LLaMA-13B
python analysis_logs.py --folder-path /path/to/logs/1f1b-
i --model-name LLaMA-13B
python analysis_logs.py --folder-path /path/to/logs/slim-
i/n=1p --model-name LLaMA-13B
python analysis_logs.py --folder-path /path/to/logs/ckpt/
zb --model-name LLaMA-13B
python analysis_logs.py --folder-path /path/to/logs/ckpt/
half --model-name LLaMA-13B
```

The output numbers include both MFU and peak memory footprints. We draw a line graph showing the MFU of different pipeline parallelism schemes across different sequence lengths.

We draw a bar graph showing the peak memory footprint of different pipeline parallelism schemes across different sequence lengths.

```
cd plots && python pp_mfu.py && python pp_mem.py
```